SISTEMAS OPERATIVOS

UNIDAD V

Contenidos

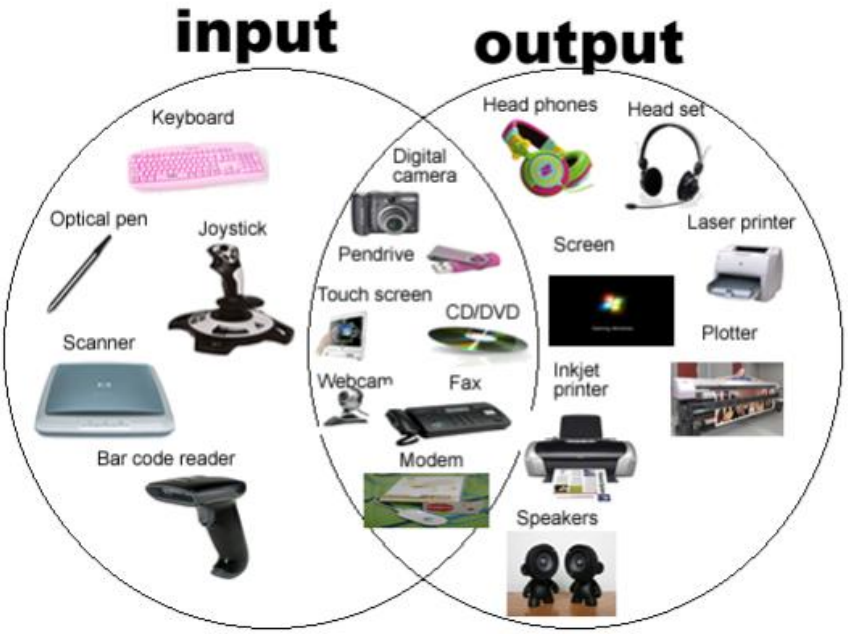
Aspectos principales del subsistema de entrada salida. 3
Introducción. 3
Principios del hardware de E/S. 4
Dispositivos de E/S. 4
Controladores de dispositivos (HW). 6
Esquemas de direccionamiento de los controladores de dispositivos (HW). 10
Acceso directo a memoria (DMA). 17
Repaso de las interrupciones. 21
Interrupciones precisas e imprecisas. 23
Fundamentos del software de E/S. 27
Objetivos del software de E/S. 27
E/S programada. 31
E/S controlada por interrupciones. 35
E/S mediante el uso de DMA. 37
Referencias y bibliografía de la sección. 39
Capas principales del software de E/S. 39
Manejadores de interrupciones. 40
Drivers de dispositivos. 42
Software de E/S independiente del dispositivo. 50
Interfaz uniforme para los controladores de software de dispositivos. 51
Uso de búfer. 52
Reporte de errores. 56
Asignación y liberación de dispositivos dedicados. 56
Tamaño de bloque independiente del dispositivo. 57
Software de E/S en espacio de usuario. 57
Referencias y bibliografía de la sección. 60
Control de energía. 60
Cuestiones de hardware. 61
Cuestiones del sistema operativo. 63
La pantalla. 63

El disco duro. 63
La CPU 64
La memoria 65
Administración térmica. 65
Administración de baterías. 66
Interfaz de drivers. 66
Cuestiones de los programas de aplicaciones 66
Referencias y bibliografía de la sección. 67

Aspectos principales del subsistema de entrada salida.

Además de proporcionar abstracciones como los procesos (e hilos), espacios de direcciones y archivos, un sistema operativo también controla todos los ***dispositivos de E/S*** (Entrada/Salida) de la computadora. Debe emitir comandos para los dispositivos, captar interrupciones y manejar errores. Adicionalmente debe proporcionar una interfaz (simple y fácil de usar) entre los dispositivos y el resto del sistema.

Hasta donde sea posible, la interfaz debe ser igual para todos los dispositivos (independencia de dispositivos). El código de E/S representa una fracción considerable del sistema operativo total.



Introducción.

Las dos tareas principales de una computadora son la E/S y el procesamiento. En muchos casos, la tarea principal es la E/S y el procesamiento es meramente incidental. Por ejemplo, cuando leemos una página web o editamos un archivo, nuestro interés inmediato es leer o introducir cierta información, no calcular una respuesta.

El papel del sistema operativo en la E/S de la computadora consiste en gestionar y controlar las operaciones y dispositivos de E/S.

El control de los dispositivos conectados a la computadora es una de las principales preocupaciones de los diseñadores de sistemas operativos. Debido a que existe una variedad tan amplia de dispositivos de E/S tanto en lo respecta a sus funciones como en cuanto a su velocidad de operación (consideremos, por ejemplo, un mouse y un disco), se necesitan diversos métodos para controlar esos dispositivos. Dichos métodos forman el subsistema de

E/S del kernel, que aísla al resto del kernel de la complejidad asociada con la gestión de los dispositivos de E/S.

En el campo de la tecnología de dispositivos de E/S se experimentan dos tendencias que están en conflicto mutuo. De un lado, podemos ver una creciente estandarización de las interfaces software y hardware; esta tendencia nos ayuda a incorporar generaciones mejoradas de dispositivos dentro de las computadoras de sistemas operativos existentes. Por otro lado, podemos ver una variedad cada vez más amplia de dispositivos de E/S; algunos nuevos dispositivos son tan distintos de los dispositivos anteriores que constituye todo un desafío incorporarlos en las computadoras y los sistemas operativos. Este desafío se afronta mediante una combinación de técnicas: hardware y software.

Los elementos básicos del hardware de E/S, como los puertos, buses y controladores de dispositivo, permiten integrar una amplia variedad de dispositivos de E/S. Para encapsular los detalles y las peculiaridades de diferentes tipos de dispositivos, el kernel de un sistema operativo se estructura de modo que haga uso de módulos específicos controladores de dispositivos.

Los controladores de dispositivo presentan al subsistema de E/S una interfaz uniforme de acceso a los dispositivos, de forma parecida a como las llamadas al sistema proporcionan una interfaz estándar entre la aplicación y el sistema operativo.

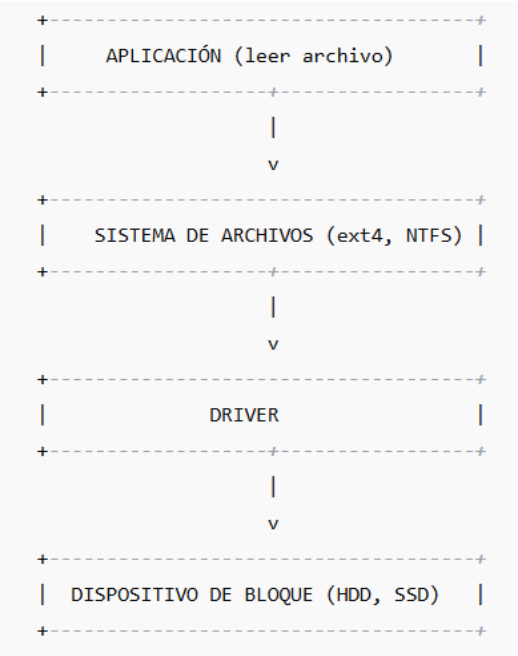
Principios del hardware de E/S.

Distintas personas ven el hardware de E/S de diferentes maneras. Los ingenieros electrónicos lo ven en términos de chips, cables, fuentes de poder, motores y todos los demás componentes físicos que constituyen el hardware. Los programadores ven la interfaz que se presenta al software: los comandos que acepta el hardware, las funciones que lleva a cabo y los errores que se pueden reportar. Sin embargo, la programación de muchos dispositivos de E/S a menudo está íntimamente conectada con su operación interna.

Dispositivos de E/S.

Los dispositivos de E/S se pueden dividir básicamente en dos categorías: ***dispositivos de bloque*** y ***dispositivos de carácter***.

Un dispositivo de bloque almacena información en bloques de tamaño fijo, cada uno con su propia dirección. Los tamaños de bloque comunes varían desde 512 bytes hasta 32,768 bytes. Todas las transferencias se realizan en unidades de uno o más bloques completos (consecutivos). La propiedad esencial de un dispositivo de bloque es que es posible leer o escribir cada bloque de manera independiente de los demás. Los discos duros, CD-ROMs y memorias USBs son dispositivos de bloque comunes.



Como resultado de un análisis más detallado, se puede concluir que no está bien definido el límite entre los dispositivos que pueden direccionarse por bloques y los que no se pueden direccionar así.

Todos concuerdan en que un disco es un dispositivo direccionable por bloques, debido a que no importa dónde se encuentre el brazo en un momento dado, siempre será posible buscar en otro cilindro y después esperar a que el bloque requerido gire debajo de la cabeza.

El otro tipo de dispositivo de E/S es el ***dispositivo de carácter***. Un dispositivo de carácter envía o acepta un flujo de caracteres, sin importar la estructura del bloque. No es direccionable y no tiene ninguna operación de búsqueda. Las impresoras, las interfaces de red, los ratones (para señalar) y la mayoría de los demás dispositivos que no son parecidos al disco se pueden considerar como dispositivos de carácter.

Este esquema de clasificación no es perfecto. Algunos dispositivos simplemente no se adaptan.

Dispositivo	¿Por bloques?	¿Por carácter?	Explicación
Disco duro (HDD/SSD)	✅ Sí	❌ No	Acceso por sectores o bloques definidos.
Teclado	❌ No	✅ Sí	Flujo continuo de caracteres, sin bloques.
CD-ROM / DVD	✅ Sí	⚠️ A veces ambos	Puede leerse por bloques, pero algunas interfaces lo tratan como flujo.
Memoria flash (pendrive)	✅ Sí	⚠️ A veces ambos	Algunos controladores permiten acceso secuencial tipo carácter.

Por ejemplo, los relojes no son direccionables por bloques. Tampoco generan ni aceptan flujos de caracteres. Todo lo que hacen es producir interrupciones a intervalos bien definidos. Las pantallas por asignación de memoria tampoco se adaptan bien al modelo.

Aun así, el modelo de dispositivos de bloque y de carácter es lo bastante general como para poder utilizarlo como base para hacer que parte del sistema operativo que lidia con los dispositivos de E/S sea independiente. Por ejemplo, el sistema de archivos sólo se encarga de los dispositivos de bloque abstractos y deja la parte dependiente de los dispositivos al software de bajo nivel.

Los dispositivos de E/S cubren un amplio rango de velocidades, lo cual impone una presión considerable en el software para obtener un buen desempeño sobre muchos órdenes de magnitud en las velocidades de transferencia de datos.

Dispositivo	Velocidad de transferencia de datos
Teclado	10 bytes/seg
Ratón	100 bytes/seg
Módem de 56K	7 KB/seg
Escáner	400 KB/seg
Cámara de video digital	3.5 MB/seg
802.11g inalámbrico	6.75 MB/seg
CD-ROM de 52X	7.8 MB/seg
Fast Ethernet	12.5 MB/seg
Tarjeta Compact Flash	40 MB/seg
FireWire (IEEE 1394)	50 MB/seg
USB 2.0	60 MB/seg
Red SONET OC-12	78 MB/seg
Disco SCSI Ultra 2	80 MB/seg
Gigabit Ethernet	125 MB/seg
Unidad de disco SATA	300 MB/seg
Cinta de Ultrium	320 MB/seg
Bus PCI	528 MB/seg

Ilustración 1 - Velocidades de transferencia de datos comunes de algunos dispositivos, redes y buses.

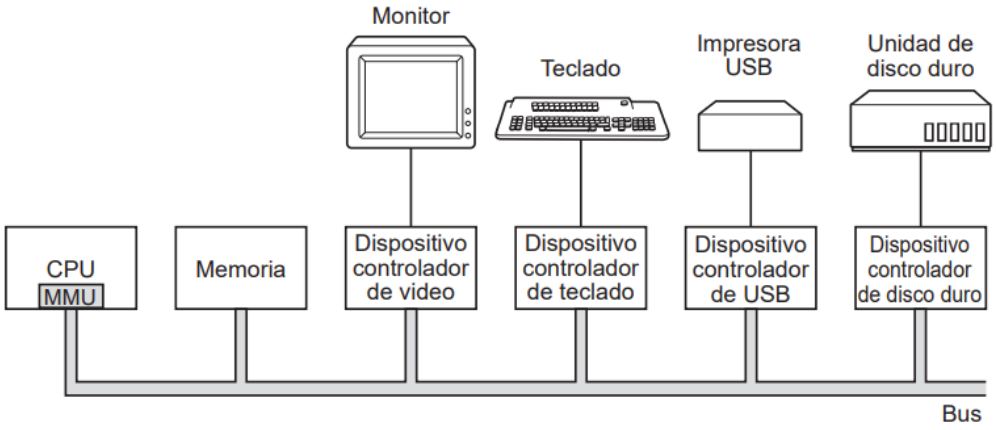
La figura anterior muestra las velocidades de transferencia de datos de algunos dispositivos comunes. La mayoría de estos dispositivos tienden a hacerse más rápidos a medida que pasa el tiempo.

Controladores de dispositivos (HW).

A veces, las unidades de E/S consisten en un componente mecánico y un componente electrónico. A menudo es posible separar las dos porciones para proveer un diseño más modular y general.

El componente electrónico se llama ***controlador de dispositivo*** o ***adaptador***. En las computadoras personales, comúnmente tiene la forma de un chip en la tarjeta principal o una tarjeta de circuito integrado que se puede insertar en una ranura de expansión.

El componente mecánico es el dispositivo en sí. Este arreglo se muestra en la figura siguiente.



Los controladores de dispositivo adoptan diversas formas físicas. Por ejemplo, una tarjeta controladora por lo general contiene un conector, en el que se puede conectar un cable que conduce al dispositivo en sí.

Muchos controladores pueden manejar dos, cuatro o inclusive ocho dispositivos idénticos. Si la interfaz entre el controlador y el dispositivo es estándar, ya sea un estándar oficial ANSI, IEEE o ISO, o un estándar de facto, entonces las empresas pueden fabricar controladores o dispositivos que se adapten a esa interfaz. Por ejemplo, muchas empresas fabrican unidades de disco que coinciden con la interfaz IDE, SATA, SCSI, USB o FireWire (IEEE 1394).

Tarea / Responsabilidad	Componente Electrónico (Controlador)	Componente Mecánico (Dispositivo en sí)
Interfaz con el Kernel	**Sí:** Ofrece registros legibles/escribibles por el *driver*.	**No:** No se comunica directamente con la CPU.
Operación Física	Controla voltajes, tiempos de motores y lógica digital.	Gira discos, mueve brazos, emite láseres o activa píxeles.

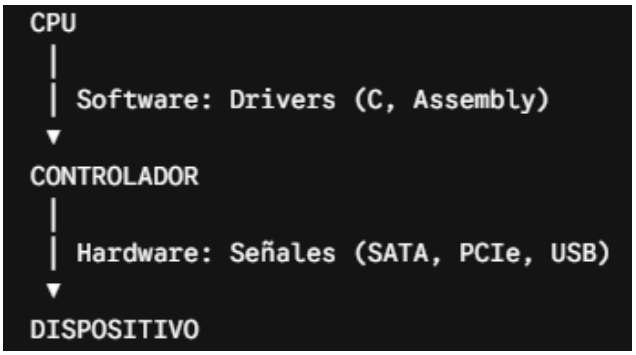
La interfaz entre el controlador y el dispositivo es a menudo de muy bajo nivel ya que maneja señales eléctricas, temporización y comandos de muy bajo nivel (pulsos, estados de líneas,

sectores magnéticos). Por otro lado, la interfaz entre la CPU y el controlador de dispositivo es de un nivel más alto (registros, direcciones, comandos en bloque).



Normalmente la interfaz entre la CPU y el controlador de dispositivo se maneja mediante código de programación ya que es una una ***interfaz lógica y programable***. Los drivers de dispositivo (software) están escritos principalmente en C y C++, e históricamente en ensamblador (assembly) para las rutinas críticas de bajo nivel. En sistemas modernos (como el kernel de Linux), también se utiliza Rust.

Por su parte, la interfaz entre el controlador y el dispositivo no está escrita en un lenguaje de programación, sino que se define mediante especificaciones y protocolos de hardware ya que es una ***interfaz física y electrónica***. La interacción real con el dispositivo se realiza mediante señales eléctricas, ópticas o electromagnéticas bajo estándares estrictos de hardware (como SATA, NVMe, USB, PCIe, I2C o SPI). Estas señales transmiten pulsos de voltaje, patrones de sincronización de reloj (clock) y secuencias de bits de bajo nivel para mover cabezales, activar celdas flash o leer sensores.



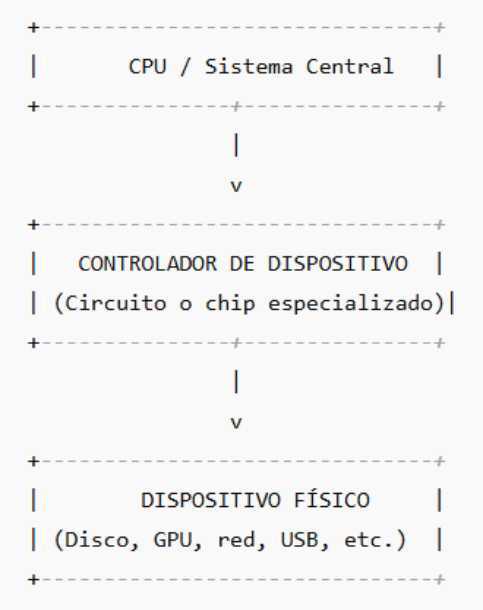
Por ejemplo, se podría dar formato a un disco con 10,000 sectores de 512 bytes por pista. Sin embargo, lo que en realidad sale del disco es un flujo serial de bits, empezando con un ***preámbulo***, después los 4096 bits en un sector y por último una suma de comprobación, también conocida como ***Código de Corrección de Errores (ECC)***. El preámbulo se escribe cuando se da formato al disco y contiene el cilindro y número de sector, el tamaño del sector y datos similares, así como información de sincronización.

El trabajo del controlador es convertir el flujo de bits serial en un bloque de bytes y realizar cualquier corrección de errores necesaria. Por lo general, primero se ensambla el bloque de bytes, bit por bit, en un búfer dentro del controlador. Después de haber verificado su suma de comprobación y de que el bloque se haya declarado libre de errores, puede copiarse a la memoria principal.

Función	Descripción
🧠 **Interpretar órdenes**	Recibe comandos del procesador (por ejemplo: "leer sector 10 del disco") y los traduce en señales eléctricas que el dispositivo entiende.
🔁 **Transferir datos**	Mueve la información entre el dispositivo y la memoria principal (a veces usando **DMA**, acceso directo a memoria).
🔔 **Generar interrupciones**	Notifica a la CPU cuando una operación ha terminado (por ejemplo, "ya se imprimió la página").
⚙️ **Controlar estados**	Supervisa y reporta el estado del dispositivo (ocupado, libre, error, etc.).

El controlador para un viejo monitor CRT[1] también funciona como un dispositivo de bits en serie a un nivel igual de bajo. Lee los bytes que contienen los caracteres a mostrar de la memoria y genera las señales utilizadas para modular el haz del CRT para hacer que escriba en la pantalla. El controlador también genera las señales para que el haz del CRT realice un retrazado horizontal después de haber terminado una línea de exploración, así como las señales para hacer que realice un retrazado vertical después de haber explorado toda la pantalla.

[1] Es un chip que controla la señal de video que se envía a un tubo de rayos catódicos (CRT), el tipo de pantalla que solían ser muy comunes en computadoras y televisores.



Si no fuera por el controlador del CRT, el programador del sistema operativo tendría que programar de manera explícita la exploración análoga del tubo. Con el controlador, el sistema operativo inicializa el controlador con unos cuantos parámetros, como el número de caracteres o píxeles por línea y el número de líneas por pantalla, y deja que el controlador se encargue de manejar el haz.

Las pantallas TFT planas funcionan de manera diferente, pero son igualmente complicadas.

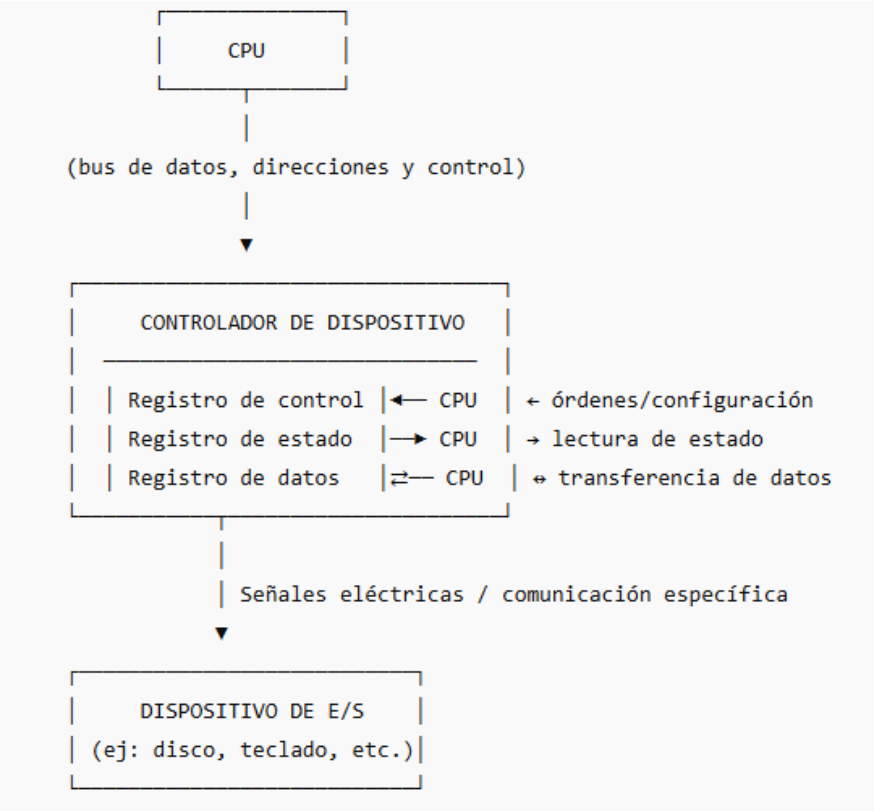
Dispositivo	Controlador de hardware	Función principal
Disco duro	Controlador de disco (SATA, NVMe)	Coordina lectura y escritura de sectores.
Tarjeta de video (GPU)	Controlador gráfico	Procesa y muestra información visual.

Esquemas de direccionamiento de los controladores de dispositivos (HW).

Cada controlador tiene unos cuantos ***registros*** que se utilizan para comunicarse con la CPU. Al escribir en ellos, el sistema operativo puede hacer que el dispositivo envíe o acepte datos, se encienda o se apague, o realice cualquier otra acción.

Al leer de estos registros, el sistema operativo puede conocer el estado del dispositivo, si está preparado o no para aceptar un nuevo comando, y sigue procediendo de esa manera.

Además de los registros de control, muchos dispositivos tienen un búfer de datos que el sistema operativo puede leer y escribir. Por ejemplo, una manera común para que las computadoras muestren píxeles en la pantalla es tener una RAM de video, la cual es básicamente sólo un búfer de datos disponible para que los programas o el sistema operativo escriban en él.



Cuando se necesita ser más preciso, es posible diferenciar de varios tipos de registros de controlador de dispositivos. En lugar de utilizar el término “registro de control” de forma general, se pueden diferenciar varios tipos:

1. **Registro de estado (status register):**
   - Indica el estado del dispositivo (por ejemplo: “ocupado”, “listo”, “error”, etc.).
2. **Registro de control (control register):**
   - La CPU escribe aquí órdenes o configuraciones (por ejemplo: “iniciar lectura”, “activar interrupciones”, etc.).
3. **Registro de datos (data register):**
   - Sirve para transferir los datos entre el controlador y la CPU (por ejemplo, el byte que se lee o escribe en el dispositivo).

De todo esto surge la cuestión acerca de cómo se comunica la CPU con los registros de control y los búferes de datos de los dispositivos. Existen dos alternativas. En el primer método, a cada registro de control se le asigna un número de ***puerto de E/S***, un entero de 8 o 16 bits. A esta técnica se la conoce como ***E/S por asignación de puertos*** (Port-Mapped I/O).

El conjunto de todos los puertos de E/S forma el ***espacio de puertos de E/S*** y está protegido de manera que los programas de usuario ordinarios no puedan utilizarlo (sólo el sistema

operativo puede). Es un conjunto de direcciones especiales a las que la CPU puede acceder para comunicarse con controladores de dispositivos.

Mediante el uso de una instrucción especial de E/S tal como:

```
IN REG,PUERTO
```

La CPU puede leer el registro de control PUERTO y almacenar el resultado en el registro de la CPU llamado REG. De manera similar, mediante el uso de:

```
OUT PUERTO, REG
```

La CPU puede escribir el contenido de REG en un registro de control. La mayoría de las primeras computadoras, incluyendo a casi todas las mainframes, como la IBM 360 y todas sus sucesoras, trabajaban de esta manera.

En este esquema, los espacios de direcciones para la memoria y la E/S son distintos, como se muestra en la parte *a* de la figura siguiente.

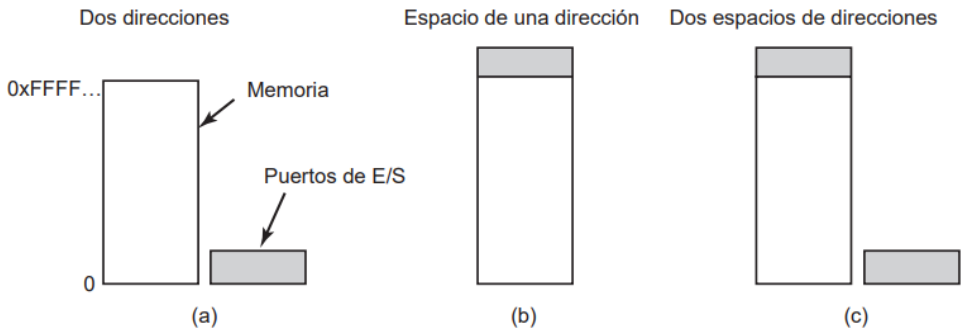


Ilustración 2 - (a) Espacio separado de E/S y memoria. (b) E/S por asignación de memoria. (c) Híbrido.

El segundo método (parte *b* de la figura), es asignar todos los registros de control al espacio de memoria. A cada registro de control se le asigna una dirección de memoria única a la cual no hay memoria asignada. Este sistema se conoce como **E/S con asignación de memoria** (*mapped-memory*). Por lo general, las direcciones asignadas se encuentran en la parte superior del espacio de direcciones.

En la parte *c* de la figura se muestra un esquema híbrido, con búferes de datos de E/S por asignación de memoria y puertos de E/S separados para los registros de control. El Pentium utilizaba esta arquitectura, donde las direcciones de 640K a 1M se reservan para los búferes de datos de los dispositivos en las IBM PC compatibles, además de los puertos de E/S de 0 a 64K.

En todos los casos, cuando la CPU desea leer una palabra, ya sea de la memoria o de un puerto de E/S, coloca la dirección que necesita en las líneas de dirección del bus y después impone una señal ***READ*** en una línea de control del bus. Se utiliza una segunda línea de señal para indicar si se necesita espacio de E/S o de memoria. Si es espacio de memoria, ésta responde a la petición. Si es espacio de E/S, el dispositivo de E/S responde a la petición. Si sólo hay espacio

de memoria (como en la parte *b* de la figura), todos los módulos de memoria y todos los dispositivos de E/S comparan las líneas de dirección con el rango de direcciones a las que dan servicio. Si la dirección está en su rango, responde a la petición.

Como ninguna dirección se asigna tanto a la memoria como a un dispositivo de E/S, no hay ambigüedad ni conflicto.

Sistemas operativos como ***Linux*** utilizan de forma nativa un esquema híbrido, ya que debe adaptarse a las posibilidades y limitaciones de la arquitectura del procesador sobre el que se está ejecutando. De esta forma, se elige mayoritariamente la E/S con asignación de memoria (Memory-Mapped I/O) para controlar los dispositivos de hardware de alto rendimiento, aunque por cuestiones de arquitectura (como en las PCs con procesadores x86) el kernel también da soporte a la E/S por asignación de puertos (Port-Mapped I/O).

¿Cómo se comunican?	¿Cuándo lo usa Linux?	Características
Por Asignación de Memoria	Dispositivos modernos y rápidos (como placas de video o discos NVMe).	Es el método principal. Usa instrucciones comunes de memoria a máxima velocidad.
Por Asignación de Puertos	Componentes antiguos o muy simples (como el teclado clásico o el temporizador).	Es el soporte heredado.

Los dos esquemas para direccionar los controladores tienen distintos puntos fuertes y débiles.

Vamos a empezar con las ventajas de la E/S por asignación de memoria. En primer lugar, si se necesitan instrucciones de E/S especiales para leer y escribir en los registros de control, para acceder a ellos se requiere el uso de código ensamblador, ya que no hay forma de ejecutar una instrucción ***IN*** o ***OUT*** en C o C++. Al llamar a dicho procedimiento se agrega sobrecarga para controlar la E/S.

```
C/C++ Program
     |
     v
Ensamblador (IN / OUT)
     |
     v
Puerto del dispositivo
     |
     v
Dispositivo físico
```

Ilustración 3 - Cómo C usa ensamblador en la E/S por asignación de puertos.

En contraste, con la E/S por asignación de memoria los registros de control de dispositivos son sólo variables en memoria y se pueden direccionar en C de la misma forma que cualquier otra variable.

Así, con la E/S por asignación de memoria, un controlador de dispositivo de E/S puede escribirse completamente en C. Sin la E/S por asignación de memoria se requiere cierto código ensamblador.

Método	Instrucciones especiales	Acceso desde C/C++	Comentario
Memory-Mapped I/O	No	Sí	Se accede como memoria normal
Port-Mapped I/O	Sí (`IN` / `OUT`)	No directamente	Requiere ensamblador o funciones especiales, con sobrecarga

En segundo lugar, con la E/S por asignación de memoria no se requiere un mecanismo de protección especial para evitar que los procesos realicen operaciones de E/S. Todo lo que el sistema operativo tiene que hacer es abstenerse de colocar esa porción del espacio de direcciones que contiene los registros de control en el espacio de direcciones virtuales de cualquier usuario. Mejor aún, si cada dispositivo tiene sus registros de control en una página distinta del espacio de direcciones, el sistema operativo puede proporcionar a un usuario el control sobre dispositivos específicos, pero no el de los demás, con sólo incluir las páginas deseadas en su tabla de páginas.

Dicho esquema permite que se coloquen distintos controladores de dispositivos en diferentes espacios de direcciones, lo cual no sólo reduce el tamaño del kernel, sino que también evita que un controlador interfiera con los demás.

En tercer lugar, con la E/S por asignación de memoria, cada instrucción que puede hacer referencia a la memoria también puede hacerla a los registros de control. Por ejemplo, si hay una instrucción llamada ***TEST*** que evalúe si una palabra de memoria es 0, también se puede utilizar para evaluar si un registro de control es 0, lo cual podría ser la señal de que el dispositivo está inactivo y en posición de aceptar un nuevo comando.

En el diseño de computadoras, casi todo implica tener que hacer concesiones, y acá también es el caso. La E/S por asignación de memoria también tiene sus desventajas. En primer lugar, la mayoría de las computadoras actuales tienen alguna forma de colocar en caché las palabras de memoria. En estos casos, el hardware debe ser capaz de deshabilitar la caché en forma selectiva; por ejemplo, por página. Esta característica agrega una complejidad adicional al hardware y al sistema operativo, que tiene que encargarse de la caché selectiva.

En segundo lugar, si sólo hay un espacio de direcciones, entonces todos los módulos de memoria y todos los dispositivos de E/S deben examinar todas las referencias a memoria para ver a cuáles debe responder. Si la computadora tiene un solo bus, como en la parte *a* de la siguiente figura es simple hacer que todos analicen cada dirección.

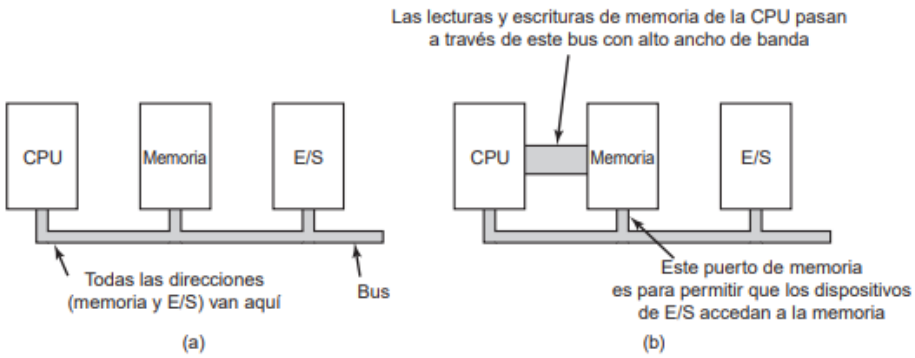


Ilustración 4 - (a) Arquitectura con un solo bus. (b) Arquitectura con bus dual.

Sin embargo, la tendencia en las computadoras personales modernas es tener un bus de memoria dedicado de alta velocidad, como se muestra en la parte *b*, una propiedad que también se encuentra incidentalmente en las mainframes.

Este bus está ajustado para optimizar el rendimiento de la memoria sin sacrificarse por el bienestar de los dispositivos de E/S lentos.

El problema de tener un bus de memoria separado en equipos con asignación de memoria es que los dispositivos no tienen forma de ver las direcciones de memoria a medida que recorren el bus de memoria, por lo que no tienen manera de responderles. De nuevo es necesario tomar medidas especiales para hacer que la E/S por asignación de memoria funcione en un sistema con varios buses.

Una posibilidad es enviar primero todas las referencias de memoria a la memoria; si ésta no responde, entonces la CPU prueba los otros buses. Este diseño puede funcionar, pero requiere una complejidad adicional del hardware.

Un segundo posible diseño es colocar un dispositivo husmeador en el bus de memoria para pasar todas las direcciones presentadas a los dispositivos de E/S potencialmente interesados.

El problema aquí es que los dispositivos de E/S tal vez no puedan procesar las peticiones a la velocidad que puede procesarlas la memoria.

Un tercer posible diseño, que es el utilizado en la configuración del Pentium de la figura siguiente, es filtrar las direcciones en el chip del puente PCI.

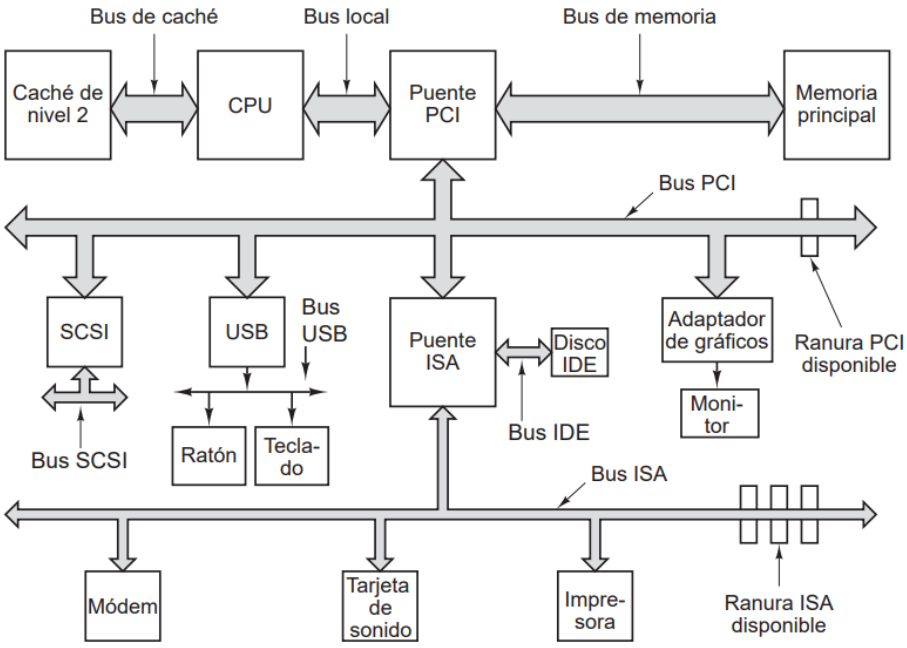


Ilustración 5 - La estructura de un sistema Pentium extenso.

Este chip contiene registros de rango que se cargan previamente en tiempo de inicio del sistema. Por ejemplo, el rango de 640K a 1M se podría marcar como un rango que no tiene memoria; las direcciones ubicadas dentro de uno de los rangos así marcados se reenvían al bus PCI, en vez de reenviarse a la memoria.

La desventaja de este esquema es la necesidad de averiguar en tiempo de inicio del sistema qué direcciones de memoria no lo son en realidad. Por ende, cada esquema tiene argumentos a favor y en contra, por lo que los sacrificios y las concesiones son inevitables.

Tipo de Arquitectura	Ventaja Principal	Desventaja en E/S
Un solo Bus Global	**Muy simple:** Los controladores de E/S están conectados directamente al mismo bus que la CPU y la memoria principal.	**Lento:** La memoria principal se ralentiza para acoplarse a la velocidad de los dispositivos de E/S.
Buses Separados (Moderno)	**Muy rápido:** La memoria principal opera a máxima velocidad en su propio bus dedicado, sin interferencias.	**Complejo:** Requiere chips puente para desviar los datos y configuraciones de software para evitar lecturas falsas de la caché.

En la arquitectura de computadoras, chips de puente similares al descripto anteriormente, generalmente se conocen con el nombre de Puente Norte (***Northbridge***). Es un componente

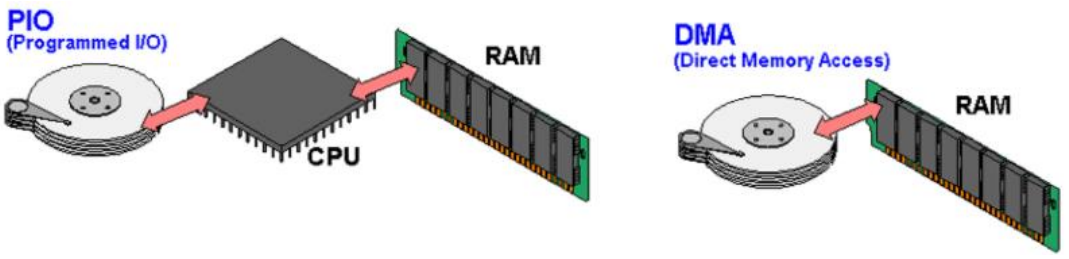
electrónico que interconecta el bus de memoria local de alta con el bus de E/S (el bus PCI en el ejemplo Pentium).

Sin embargo, al analizar la evolución del hardware moderno, en los procesadores actuales las funciones de este chip, en muchos casos se han integrado directamente dentro de la propia CPU. Cuando está integrado, se lo suele denominar técnicamente como ***controlador de memoria integrado (IMC)***, también es conocido como ***System Agent*** (Intel) o ***I/O Die*** (AMD), dependiendo del fabricante.

Criterio Técnico	Chip de Puente Independiente (Northbridge)	Función Integrada en la CPU (Agente del Sistema)
Ubicación Física	Chip separado instalado afuera, en la placa madre.	Integrado directamente dentro del mismo silicio de la CPU.
Latencia (Retraso)	**Mayor:** Los datos deben viajar por las pistas físicas de la placa madre.	**Mínima:** Los datos viajan por conexiones internas microscópicas.

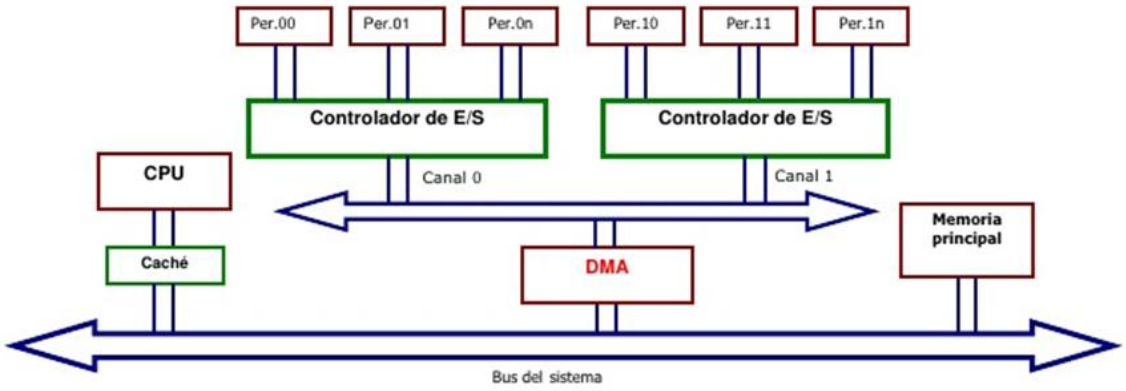
Acceso directo a memoria (DMA).

Sin importar que una CPU tenga o no E/S por asignación de memoria, necesita direccionar los controladores de dispositivos para intercambiar datos con ellos. La CPU puede solicitar datos de un controlador de E/S un bit a la vez, pero al hacerlo se desperdicia el tiempo de la CPU, por lo que a menudo se utiliza un esquema distinto, conocido como ***DMA*** (Acceso Directo a Memoria).



El sistema operativo sólo puede utilizar DMA si el hardware tiene un controlador de DMA, que la mayoría de los sistemas tienen. Algunas veces este controlador está integrado a los controladores de disco y otros controladores, pero dicho diseño requiere un controlador de DMA separado para cada dispositivo.

Otra alternativa, es que haya un solo controlador de DMA disponible (por ejemplo, en la tarjeta principal) para regular las transferencias a varios dispositivos, a menudo en forma concurrente.



En la actualidad, y siguiendo la evolución del hardware, el controlador DMA tradicional como un chip único y separado en la tarjeta principal prácticamente ha desaparecido.

Como observamos anteriormente, en los procesadores modernos (como Intel Core o AMD Ryzen), las funciones del antiguo Puente Norte (Northbridge) se mudaron al interior del procesador en una zona llamada ***controlador de memoria integrado*** (IMC).

Dentro de este bloque se encuentra un controlador ***DMA*** de altísimo rendimiento que está directamente conectado al IMC y a las líneas PCIe nativas del procesador.

Esto permite que los dispositivos de alta velocidad (como una placa de video o un disco NVMe) transfieran gigabytes de datos hacia la memoria principal de la forma más directa y con la menor latencia posible.

Sin embargo, para los componentes de alto rendimiento, se consolidó la ya mencionada alternativa de que cada dispositivo tenga con su propio DMA (a menudo llamado Bus Mastering). Por ejemplo, algunas unidades NVMe o tarjetas de red de 10 Gbps, cuentan con un chip DMA diseñado a medida para sus velocidades.

Sin importar cuál sea su ubicación física, el controlador de DMA tiene acceso al bus del sistema de manera independiente de la CPU, como se muestra en la figura siguiente.

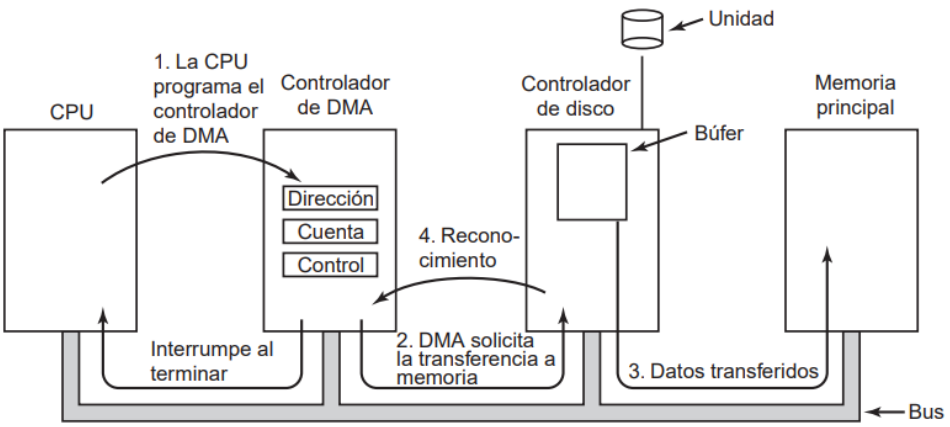


Ilustración 6 - Operación de una transferencia de DMA.

Contiene varios registros en los que la CPU puede escribir y leer; éstos incluyen un registro de dirección de memoria, un registro contador de bytes y uno o más registros de control.

Los registros de control especifican el puerto de E/S a utilizar, la dirección de la transferencia (si se va a leer del dispositivo de E/S o se va a escribir en él), la unidad de transferencia (un byte a la vez o una palabra a la vez), y el número de bytes a transferir en una ráfaga.

Para explicar el funcionamiento del DMA, veamos primero cómo ocurren las lecturas de disco cuando no se utiliza DMA. Primero, el controlador de disco lee el bloque (uno o más sectores) de la unidad en forma serial, bit por bit, hasta que se coloca todo el bloque completo en el búfer interno del controlador. Después calcula la suma de comprobación para verificar que no hayan ocurrido errores de lectura. Después, el controlador produce una interrupción. Cuando el sistema operativo empieza a ejecutarse, puede leer el bloque de disco del búfer del controlador, un byte o una palabra a la vez, mediante la ejecución de un ciclo en el que cada iteración se lee un byte o palabra de un registro de dispositivo controlador y se almacena en la memoria principal.

Cuando se utiliza DMA, el procedimiento es distinto. Primero, la CPU programa el controlador de DMA, para lo cual establece sus registros de manera que sepa qué debe transferir y a dónde (paso 1 en la figura anterior). También emite un comando al controlador de disco para indicarle que debe leer datos del disco en su búfer interno y verificar la suma de comprobación. Cuando hay datos válidos en el búfer del controlador de disco, puede empezar el DMA.

El controlador de DMA inicia la transferencia enviando una petición de lectura al controlador de disco mediante el bus (paso 2 de la figura anterior). Esta petición de lectura se ve como cualquier otra petición de lectura, por lo que el controlador de disco no sabe ni le importa si vino de la CPU o de un controlador de DMA. Por lo general, la dirección de memoria en la que se va a escribir está en las líneas de dirección del bus, por lo que cuando el controlador de disco obtiene la siguiente palabra de su búfer interno, sabe dónde escribir.

La escritura en memoria es otro ciclo de bus estándar (paso 3).

Cuando se completa la escritura, el controlador de disco envía una señal de reconocimiento al controlador de DMA, también a través del bus (paso 4). El controlador de DMA incrementa a continuación la dirección de memoria a utilizar y disminuye la cuenta de bytes. Si la cuenta de bytes es aún mayor que 0, se repiten los pasos del 2 al 4 hasta que la cuenta llega a 0. En ese momento, el controlador de DMA interrumpe la CPU para hacerle saber que la transferencia está completa.

Cuando el sistema operativo se inicia, no tiene que copiar el bloque de disco en la memoria; ya se encuentra ahí.

Los controladores de DMA varían considerablemente en cuanto a su sofisticación. Los más simples manejan una transferencia a la vez, como se describió antes. Los más complejos se pueden programar para manejar varias transferencias a la vez. Dichos controladores tienen varios conjuntos de registros internos, uno para cada canal. La CPU empieza por cargar cada conjunto de registros con los parámetros relevantes para su transferencia. Cada transferencia debe utilizar un controlador de dispositivo distinto. Después de transferir cada palabra (los pasos 2 al 4) en la figura anterior, el controlador de DMA decide a cuál dispositivo va a dar

servicio a continuación. Se puede configurar para utilizar un algoritmo por turno rotatorio (*round-robin*), o puede tener un diseño de esquema prioritario para favorecer a unos dispositivos sobre otros.

Puede haber varias peticiones a distintos controladores de dispositivos pendientes al mismo tiempo, siempre y cuando haya una manera ambigua de diferenciar las señales de reconocimiento. A menudo se utiliza una línea de reconocimiento distinta en el bus para cada canal de DMA por esta razón.

Muchos buses pueden operar en dos modos: el modo de una palabra a la vez y el modo de bloque. Algunos controladores de DMA también pueden operar en cualquier modo.

En el modo de una palabra a la vez, la operación es como se describe antes: el controlador de DMA solicita la transferencia de una palabra y la obtiene; si la CPU también desea el bus, tiene que esperar. El mecanismo se llama ***robo de ciclo*** debido a que el controlador de dispositivo se acerca de manera sigilosa y roba un ciclo de bus a la CPU de vez en cuando, retrasándola ligeramente.

En el modo de bloque, el controlador de DMA indica al dispositivo que debe adquirir el bus, emitir una serie de transferencias y después liberar el bus. Esta forma de operación se conoce como ***modo de ráfaga***. Es más eficiente que el robo de ciclo, debido a que la adquisición del bus requiere tiempo y se pueden transferir varias palabras con sólo una adquisición de bus.

La desventaja en comparación con el modo de ráfaga es que puede bloquear la CPU y otros dispositivos por un periodo considerable, si se está transfiriendo una ráfaga extensa.

La mayoría de los controladores de DMA utilizan direcciones físicas de memoria para sus transferencias. El uso de direcciones físicas requiere que el sistema operativo convierta la dirección virtual del búfer de memoria deseado a una dirección física, y que escriba esta dirección física en el registro de dirección del controlador de DMA. Un esquema alternativo que se utiliza en unos cuantos controladores de DMA es escribir direcciones virtuales en el controlador de DMA. Así, el controlador de DMA debe utilizar la ***MMU*** para realizar la traducción de dirección virtual a física.

Sólo en el caso en que la MMU sea parte de la memoria (es posible, pero raro) en vez de formar parte de la CPU, las direcciones virtuales se podrán colocar en el bus.

Como lo mencionamos anteriormente, antes de que pueda iniciar el DMA, el disco lee primero los datos en su búfer interno. Hay dos razones para la existencia de este bufer: en primer lugar, al utilizar el búfer interno, el controlador de disco puede verificar la suma de comprobación antes de iniciar una transferencia y si la suma de comprobación es incorrecta se señala un error y no se realiza la transferencia; la segunda es que, una vez que se inicia una transferencia de disco, los bits siguen llegando a una velocidad constante, esté listo o no el controlador para ellos.

Si el controlador tratara de escribir datos directamente en la memoria, tendría que pasar por el bus del sistema por cada palabra transferida. Si el bus estuviera ocupado debido a que algún otro dispositivo lo estuviera utilizando (por ejemplo, en modo ráfaga), el controlador tendría que esperar. Si la siguiente palabra del disco llegar antes de que se almacenara la anterior, el controlador tendría que almacenarla en algún otro lado. Si el bus estuviera muy ocupado, el controlador podría terminar almacenando muy pocas palabras y tendría que realizar también mucho trabajo de administración.

Cuando el bloque se coloca en un búfer interno, el bus no se necesita sino hasta que empieza el DMA, por lo que el diseño del controlador es mucho más simple debido a que la transferencia de DMA a la memoria no es muy estricta en cuanto al tiempo (de hecho, algunos controladores antiguos van directamente a la memoria con sólo una pequeña cantidad de uso del búfer interno, pero cuando el bus está muy ocupado, tal vez haya que terminar una transferencia con un error de desbordamiento).

No todas las computadoras utilizan DMA. El argumento en contra es que la CPU principal es a menudo más rápida que el controlador de DMA y puede realizar el trabajo con mucha mayor facilidad (cuando el factor limitante no es la velocidad del dispositivo de E/S). Si no hay otro trabajo que realizar, hacer que la CPU (rápida) tenga que esperar al controlador de DMA (lento) para terminar no tiene caso. Además, al deshacernos del controlador de DMA y hacer que la CPU realice todo el trabajo en software se ahorra dinero, lo cual es importante en las computadoras de bajo rendimiento (incrustadas o embebidas).

Repaso de las interrupciones.

En una computadora personal común, la estructura de las interrupciones es como se muestra en la figura siguiente.

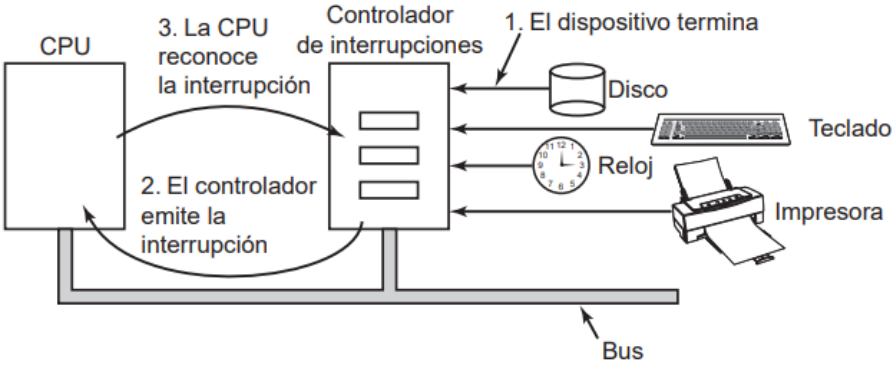


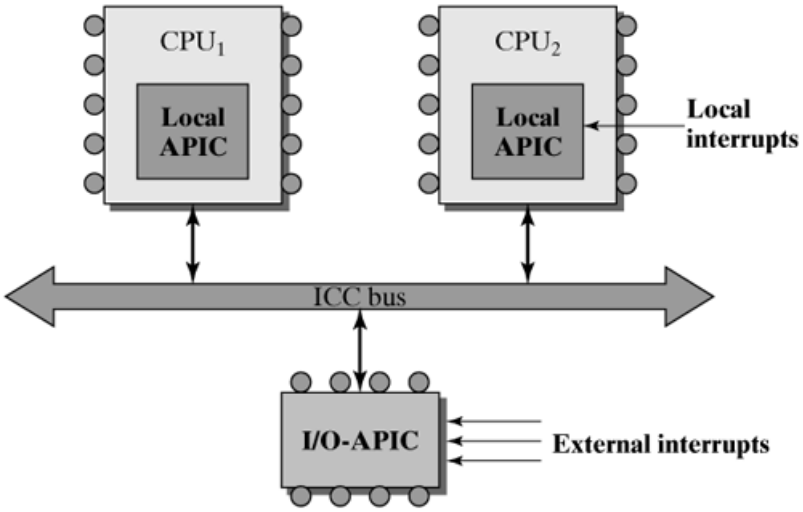
Ilustración 7 - Cómo ocurre una interrupción. Las conexiones entre los dispositivos y el controlador de interrupciones en realidad utilizan líneas de interrupción en el bus, en vez de cables dedicados.

A nivel de hardware, las interrupciones funcionan de la siguiente manera. Cuando un dispositivo de E/S ha terminado el trabajo que se le asignó, produce una interrupción (suponiendo que el sistema operativo haya habilitado las interrupciones). Para ello, impone una señal en una línea de bus que se le haya asignado. Esta señal es detectada por el chip ***controlador de interrupciones***[2], que después decide lo que debe hacer.

El controlador de interrupciones puede encontrarse ubicado o distribuido de diferentes maneras dependiendo de la arquitectura. Algunas ubicaciones típicas pueden ser:

[2] Es un dispositivo o módulo de hardware que gestiona las interrupciones de los diferentes periféricos y señales de E/S hacia la CPU. Su función principal es decidir qué interrupción debe atender primero cuando llegan varias señales al mismo tiempo.

1. **En la placa base (motherboard)**
   - En sistemas clásicos (como PC con x86) se encuentra en un **chip dedicado** llamado **PIC** (Programmable Interrupt Controller) o, en sistemas modernos, en un **APIC**.
   - Conectado directamente al **bus de la CPU**, para enviar señales de interrupción de forma rápida.
2. **Dentro del procesador**
   - En arquitecturas modernas, especialmente multiprocesador, el **controlador de interrupciones puede estar integrado dentro del CPU** (local APIC).
   - Esto reduce la latencia y permite manejar interrupciones de manera más eficiente.
3. **Distribuido**
   - En sistemas complejos, puede haber un **controlador principal** y **controladores secundarios** para distintos periféricos.



Si no hay otras interrupciones pendientes, el controlador de interrupciones procesa la interrupción de inmediato. Si hay otra en progreso, o si otro dispositivo ha realizado una petición simultánea en una línea de petición de interrupción de mayor prioridad en el bus, el dispositivo sólo se ignora por el momento. En este caso, continúa imponiendo una señal de interrupción en el bus hasta que la CPU la atiende.

Para manejar la interrupción, el controlador coloca un número en las líneas de dirección que especifican cuál dispositivo desea atención e impone una señal para interrumpir a la CPU.

La señal de interrupción hace que la CPU deje lo que está haciendo y empiece a hacer otra cosa.

El número en las líneas de dirección se utiliza como índice en una tabla llamada ***vector de interrupciones*** para obtener un nuevo contador del programa. Este contador del programa apunta al inicio del procedimiento de servicio de interrupciones correspondiente.

Por lo general, las trampas[3] e interrupciones utilizan el mismo mecanismo desde este punto en adelante, y con frecuencia comparten el mismo vector de interrupción.

La ubicación del vector de interrupción se puede determinar de manera estática (*hardwired*) en la máquina, o puede estar en cualquier parte de la memoria, con un registro de la CPU (cargado por el sistema operativo) apuntando a su origen.

Poco después de que se empieza a ejecutar, el procedimiento de servicio de interrupciones reconoce la interrupción al escribir cierto valor en uno de los puertos de E/S del controlador de interrupciones. Este reconocimiento indica al controlador que puede emitir otra interrupción. Al hacer que la CPU retrase este reconocimiento hasta que esté lista para manejar la siguiente interrupción, se pueden evitar las condiciones de competencia que involucran varias interrupciones (casi simultáneas).

Por otra parte, algunas computadoras (antiguas) no tienen un controlador de interrupciones centralizado, por lo que cada controlador de dispositivo solicita sus propias interrupciones.

El hardware siempre guarda cierta información antes de iniciar el procedimiento de servicio. La información que se guarda y el lugar en donde se guarda varía de manera considerable, entre una CPU y otra. Como mínimo se debe guardar el contador del programa para que se pueda reiniciar el proceso interrumpido. En el otro extremo, todos los registros visibles y un gran número de registros internos se pueden guardar también.

Una cuestión es dónde guardar esta información. Una opción es colocarla en registros internos que el sistema operativo pueda leer según sea necesario. Un problema derivado de este método es que entonces el controlador de interrupciones no puede recibir una señal de reconocimiento sino hasta que se haya leído toda la información potencialmente relevante, por temor a que una segunda interrupción sobrescriba los registros internos que guardan el estado. Esta estrategia conlleva tiempos inactivos más largos cuando se deshabilitan las interrupciones, y posiblemente interrupciones y datos perdidos.

Interrupciones precisas e imprecisas.

Otro problema es ocasionado por el hecho de que la mayoría de las CPUs modernas tienen muchas líneas de tuberías y a menudo son superescalares (paralelas en su interior).

En los sistemas antiguos, después de que cada instrucción terminaba de ejecutarse, el microprograma o hardware comprobaba si había una interrupción pendiente. De ser así, el contador del programa y el PSW se metían en la pila y empezaba la secuencia de interrupción. Una vez que se ejecutaba el manejador de interrupciones, se llevaba a cabo el proceso inverso, en donde el PSW antiguo y el contador del programa se sacaban de la pila y continuaba el proceso anterior.

Este modelo hace la suposición implícita de que, si ocurre una interrupción justo después de cierta instrucción, todas las instrucciones hasta (e incluyendo) esa instrucción se ha ejecutado por completo, y no se ha ejecutado ninguna instrucción después de ésta.

[3] Una trampa (trap) es una interrupción síncrona generada por el software. Las trampas son generadas por software (excepciones o llamadas al sistema), mientras que las interrupciones son generadas por hardware (eventos de E/S, por ejemplo).

En equipos antiguos, esta suposición siempre era válida. En los modernos tal vez no sea así. Para empezar, consideremos el modelo de línea de tubería de la figura siguiente.

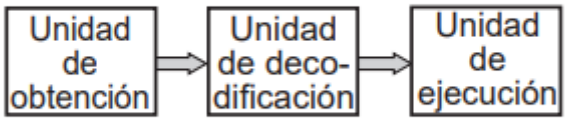


Ilustración 8 - Canalización de tres etapas.

Es necesario preguntarse qué pasa si ocurre una interrupción mientras la línea de tubería está llena (el caso usual).

Muchas instrucciones se encuentran en varias etapas de ejecución. Cuando ocurre la interrupción, el valor del contador del programa tal vez no refleje el límite correcto entre las instrucciones ejecutadas y las no ejecutadas. De hecho, muchas instrucciones pueden haberse ejecutado en forma parcial, y distintas instrucciones podrían estar más o menos completas. En esta situación, el contador del programa refleja con más probabilidad la dirección de la siguiente instrucción a obtener y meter en la línea de tubería, en vez de la dirección de la instrucción que acaba de ser procesada por la unidad de ejecución.

En una máquina superescalar, como la de la figura siguiente, las cosas empeoran.

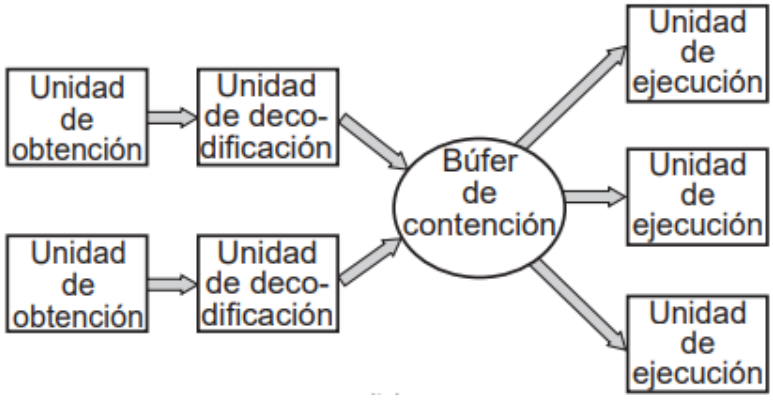


Ilustración 9 - CPU superescalar.

Las instrucciones se pueden descomponer en microoperaciones y éstas se pueden ejecutar en desorden, dependiendo de la disponibilidad de los recursos internos como las unidades funcionales y los registros. Al momento de una interrupción, algunas instrucciones iniciadas

hace tiempo tal vez no hayan empezado, y otras que hayan empezado más recientemente pueden estar casi completas. Al momento en que se señala una interrupción, puede haber muchas instrucciones en varios estados de avance, con menos relación entre ellas y el contador del programa.

Una interrupción que deja al equipo en un estado bien definido se conoce como ***interrupción precisa***. Dicha interrupción tiene cuatro propiedades:

1. El contador del programa (PC) se guarda en un lugar conocido.
2. Todas las instrucciones antes de la instrucción a la que apunta el PC se han ejecutado por completo.
3. Ninguna instrucción más allá de la instrucción a la que apunta el PC se ha ejecutado.
4. Se conoce el estado de ejecución de la instrucción a la que apunta el PC.

Observemos que no hay prohibición sobre las instrucciones más allá de la instrucción a la que apunta la PC acerca de si pueden iniciar o no. Es sólo que los cambios que realizan a los registros o la memoria se deben deshacer antes de que ocurra la interrupción. Se permite ejecutar la instrucción a la que se apunta. También se permite que no se haya ejecutado. Sin embargo, debe estar claro cuál es el caso que se aplica.

A menudo, si la interrupción es de E/S, todavía no habrá empezado. No obstante, si la interrupción es en realidad una trampa o fallo de página, entonces el PC generalmente apunta a la instrucción que ocasionó el fallo, por lo que puede reiniciarse después.

La situación de la figura siguiente ilustra una interrupción precisa.

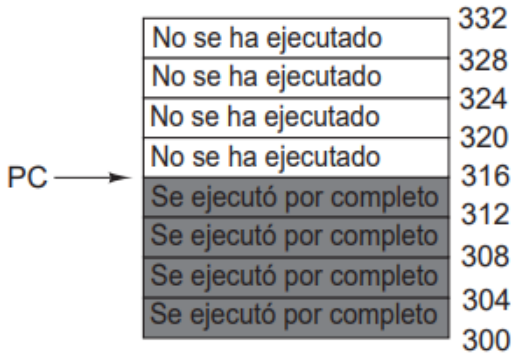


Ilustración 10 - Una interrupción precisa.

Todas las instrucciones hasta el contador del programa (316) se han completado, y ninguna más allá de estas ha iniciado (o se ha rechazado para deshacer sus efectos).

Una interrupción que no cumple con estos requerimientos se conoce como ***interrupción imprecisa*** y hace la vida muy incómoda para el escritor del sistema operativo, que entonces tiene que averiguar lo que ocurrió y lo que aún está por ocurrir. La figura siguiente muestra una interrupción imprecisa, en donde hay distintas instrucciones cerca del contador del

programa en distintos estados de avance, y las más antiguas no necesariamente están más completas que las más recientes.

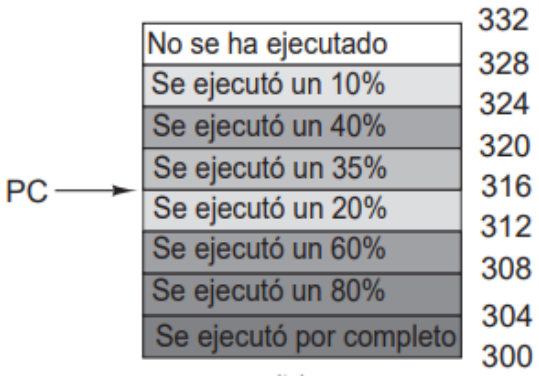


Ilustración 11 - Una interrupción imprecisa.

Las máquinas con interrupciones imprecisas por lo general vuelcan una gran cantidad de estado interno en la pila, para dar al sistema operativo la posibilidad de averiguar qué está pasando. El código necesario para reiniciar la máquina es por lo general muy complicado. Además, al guardar una cantidad extensa de información para la memoria en cada interrupción se reduce aún más la velocidad de las interrupciones y la recuperación. Esto conlleva a una irónica situación en la que se tienen CPUs superescalares muy veloces, que algunas veces no son adecuadas para el trabajo real debido a la lentitud de las interrupciones.

Algunas computadoras están diseñadas de manera que ciertos tipos de interrupciones y trampas (traps) sean precisas y otras no. Por ejemplo, hacer que las interrupciones de E/S sean precisas, pero que las trampas producidas por errores de programación fatales sean imprecisas no es tan malo, ya que no hay necesidad de intentar reiniciar un proceso en ejecución una vez que ha dividido entre cero.

Algunas máquinas tienen un bit que se puede establecer para forzar a que todas las interrupciones sean precisas. La desventaja de establecer este bit es que obliga a la CPU a registrar cuidadosamente todo lo que está haciendo, y a mantener copias “sombra” de los registros para que pueda generar una interrupción precisa en cualquier instante. Toda esta sobrecarga tiene un gran impacto sobre el rendimiento.

Algunas máquinas superescalares como la serie Pentium tienen interrupciones precisas para permitir que el software antiguo funcione correctamente. El precio a pagar por las interrupciones precisas es una lógica de interrupciones en extremo compleja dentro de la CPU necesaria para asegurar que cuando el controlador de interrupciones indique que desea ocasionar una interrupción, todas las instrucciones hasta cierto punto puedan terminar y ninguna más allá de ese punto pueda tener un efecto considerable sobre el estado de la máquina.

Acá el precio no se paga con tiempo, sino en área del chip y en la complejidad del diseño. Si no se requirieran interrupciones precisas para fines de compatibilidad con aplicaciones antiguas, esta área del chip estaría disponible para cachés en chip más grandes, haciendo a la CPU más

veloz. Por otra parte, las interrupciones hacen que el sistema operativo sea mucho más complicado y lento, por lo que es difícil saber qué método es mejor en realidad.

Fundamentos del software de E/S.

Ahora vamos a alejarnos del hardware de E/S para analizar el software de E/S, tanto los objetivos del software de E/S como las distintas formas en que se puede realizar la E/S desde el punto de vista del sistema operativo.

Objetivos del software de E/S.

Un concepto clave en el diseño del software de E/S se conoce como ***independencia de dispositivos***. Lo que significa es que debe ser posible escribir programas que puedan acceder a cualquier dispositivo de E/S sin tener que especificar el dispositivo por adelantado. Por ejemplo, un programa que lee un archivo como entrada debe tener la capacidad de leer un archivo en el disco duro, un CD-ROM, un DVD o una memoria USB sin tener que modificar el programa para cada dispositivo distinto.

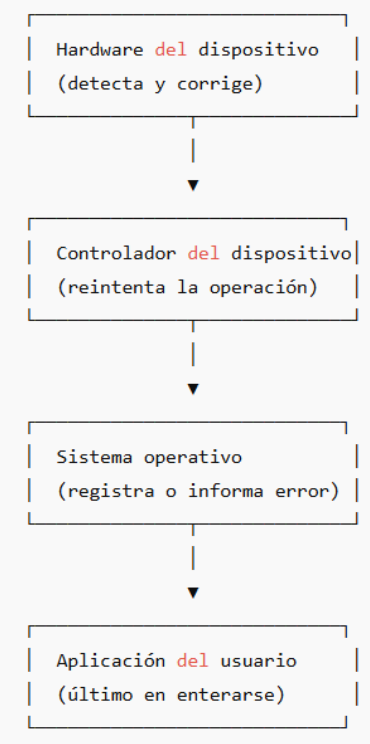
De manera similar, debe ser posible escribir un comando tal como ***sort <entrada> salida*** y hacer que funcione con datos de entrada provenientes de cualquier tipo de disco o del teclado, y que los datos de salida vayan a cualquier tipo de disco o a la pantalla.

Depende del sistema operativo encargarse de los problemas producidos por el hecho de que estos dispositivos en realidad son diferentes y requieren secuencias de comandos muy distintas para leer o escribir.

Un objetivo muy relacionado con la independencia de los dispositivos es la ***denominación uniforme***. El nombre de un archivo o dispositivo simplemente debe ser una cadena o un entero sin depender del dispositivo de ninguna forma. En UNIX, todos los discos se pueden integrar en la jerarquía del sistema de archivos de maneras arbitrarias, por lo que el usuario no necesita estar al tanto de cuál nombre corresponde a cuál dispositivo. Por ejemplo, una memoria USB se puede ***montar*** encima del directorio */usr/ast/respaldo*, de manera que al copiar un archivo a */usr/ast/respaldo/lunes*, este archivo se copie a la memoria USB. De esta forma, todos los archivos y dispositivos se direccionan de la misma forma: mediante el nombre de una ruta.

Otra cuestión importante relacionada con el software de E/S es el ***manejo de errores***. En general, los errores se deben manejar lo más cerca del hardware que sea posible. Si el controlador descubre un error de lectura, debe tratar de corregir el error por sí mismo. Si no puede, entonces el software controlador del dispositivo debe manejarlo, tal vez con sólo tratar de leer el bloque de nuevo.

Muchos errores son transitorios, como los errores de lectura ocasionados por pizcas de polvo en la cabeza de lectura, y comúnmente desaparecen si se repite la operación. Sólo si los niveles inferiores no pueden lidiar con el problema, los niveles superiores deben saber acerca de ello. En muchos casos, la recuperación de los errores se puede hacer de manera transparente a un nivel bajo, sin que los niveles superiores se enteren siquiera sobre el error.



Otra cuestión clave es la de las transferencias ***síncronas*** (de bloqueo) contra las ***asíncronas*** (controladas por interrupciones).

Tipo de transferencia	Ejemplo de dispositivo / operación	Cómo funciona
Síncrona / Bloqueante	Leer un byte de un puerto serie	La CPU espera a que llegue el dato antes de continuar.
	Escritura a un registro de hardware	La CPU espera a que se complete la operación antes de seguir.
Asíncrona / Interrupciones	Descargar archivo desde disco o red	La CPU continúa ejecutando otras tareas; el dispositivo genera una interrupción al terminar.
	Reproducción de audio o video	El buffer se llena mientras la CPU hace otras cosas; se genera interrupción cuando se necesitan más datos.

En una transferencia ***sincrónica*** la CPU inicia la operación de E/S y espera a que termine antes de continuar con cualquier otra tarea.

La mayoría de las operaciones de E/S son ***asíncronas***: la CPU inicia la transferencia y se va a hacer algo más hasta que llega la interrupción.

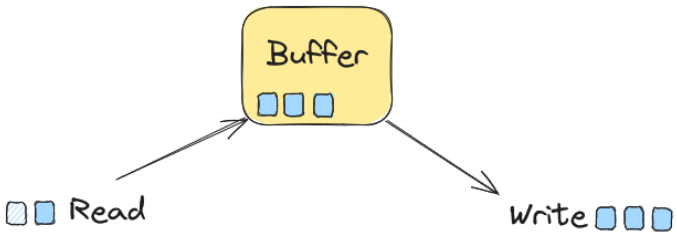
Los programas de usuario son mucho más fáciles de escribir si las operaciones de E/S son de bloqueo (síncronas): después de una llamada al sistema ***read***, el programa se suspende de manera automática hasta que haya datos disponibles en el búfer.

Depende del sistema operativo hacer que las operaciones que en realidad son controladas por interrupciones parezcan de bloqueo para los programas de usuario.

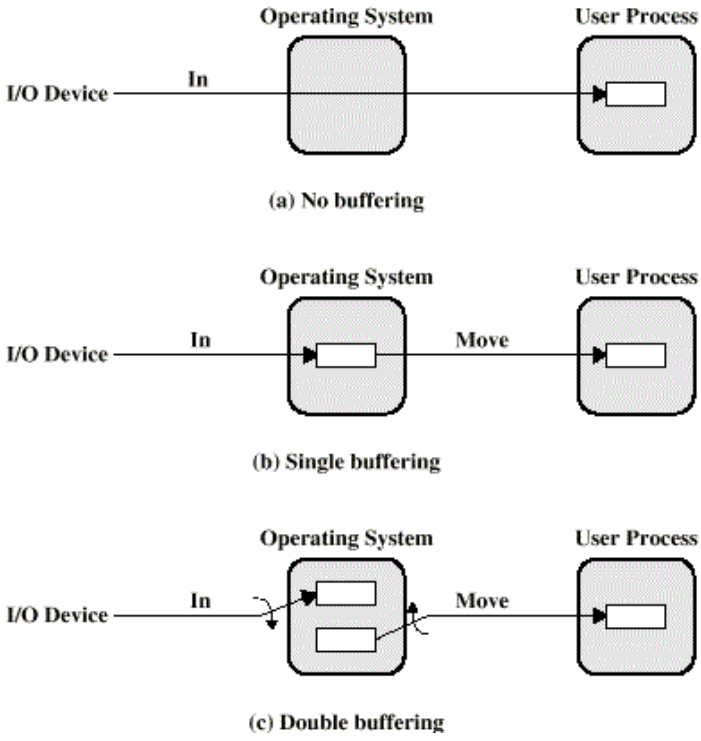
Podríamos decir que el ***DMA*** (Acceso Directo a Memoria) es un mecanismo de E/S ***asíncrono*** ya permite que un dispositivo de E/S transfiera datos directamente a la memoria sin intervención constante de la CPU.

Característica	Síncrona / Bloqueante	Asíncrona / Interrupciones
CPU espera	✅ Sí, bloqueada hasta terminar	❌ No, continúa con otras tareas
Eficiencia	Baja (CPU ociosa)	Alta (CPU aprovecha su tiempo)
Complejidad	Baja	Media/alta
Uso típico	Lecturas rápidas, simples	Operaciones lentas (disco, red)

Otra cuestión relacionada con el software de E/S es el ***uso de búfer***. A menudo los datos que provienen de un dispositivo no se pueden almacenar directamente en su destino final.



Por ejemplo, cuando un paquete llega de la red, el sistema operativo no sabe dónde colocarlo hasta que ha almacenado el paquete en alguna parte y lo examina. Además, algunos dispositivos tienen severas restricciones en tiempo real (por ejemplo, los dispositivos de audio digital), por lo que los datos se deben colocar en un búfer de salida por adelantado para desacoplar la velocidad a la que se llena el búfer, de la velocidad a la que se vacía, de manera que se eviten sub-desbordamientos de búfer. El uso de búfer involucra una cantidad considerable de copiado y a menudo tiene un importante impacto en el rendimiento de la E/S.



El concepto final que mencionaremos acá es la comparación entre los dispositivos ***compartidos*** y los dispositivos ***dedicados***.

Algunos dispositivos de E/S, como los discos, pueden ser utilizados por muchos usuarios a la vez. No se deben producir problemas debido a que varios usuarios tengan archivos abiertos en el mismo disco al mismo tiempo.

Ejemplos de dispositivos de E/S	Características principales
Disco duro / SSD	Varios procesos pueden leer o escribir simultáneamente; el SO gestiona el acceso.
Tarjeta de red (NIC)	Permite múltiples conexiones y procesos usando la red a la vez.
Dispositivo de audio (PulseAudio, PipeWire)	Mezcla sonidos de varios programas al mismo tiempo.

Otros dispositivos, como las viejas unidades de cinta o un escáner, tienen que estar dedicados a un solo usuario hasta que éste termine. Después, otro usuario puede usar ese dispositivo. Como ejemplo, si dos o más usuarios escriben bloques entremezclados al azar en la misma cinta, definitivamente no funcionará.

Al introducir los dispositivos dedicados (no compartidos) también se introduce una variedad de problemas, como los interbloqueos. De nuevo, el sistema operativo debe ser capaz de manejar los dispositivos compartidos y dedicados de una manera que evite problemas.

Tipo de dispositivo	Característica	Ejemplos
Compartido	Puede ser usado por varios procesos simultáneamente o mediante colas	Disco duro, impresora en red, tarjeta de red
Dedicado	Solo un proceso lo usa a la vez	Escáner, unidad de cinta, CD-ROM

E/S programada.

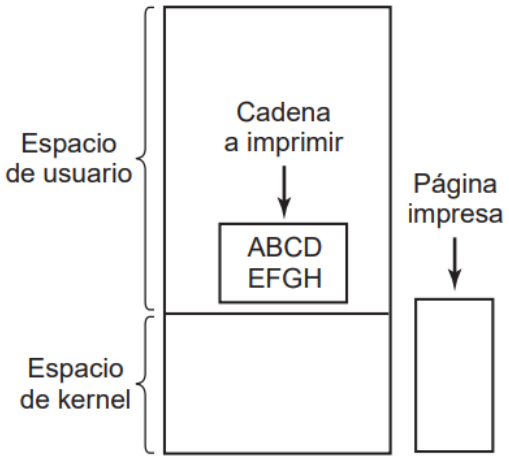
Hay tres maneras fundamentalmente distintas en que se puede llevar a cabo la E/S. Se trata de la ***E/S programada***, ***E/S controlada por interrupciones*** y ***E/S mediante el uso de DMA***.

La forma más simple de E/S es cuando la CPU hace todo el trabajo. A este método se le conoce como ***E/S programada***.

El funcionamiento básico es el siguiente:

1. El CPU **envía una orden** al dispositivo (por ejemplo, "lee un dato").
2. El CPU **espera** (consulta constantemente el estado del dispositivo).
3. Cuando el dispositivo está listo, el CPU **lee o escribe los datos**.
4. El CPU **continúa con el siguiente paso** del programa.

Es más simple ilustrar la E/S programada por medio de un ejemplo. Consideremos un proceso de usuario que desea imprimir la cadena de ocho caracteres "ABCDEFGH" en la impresora. Primero ensambla la cadena en un búfer en espacio de usuario, como se muestra en la figura siguiente.

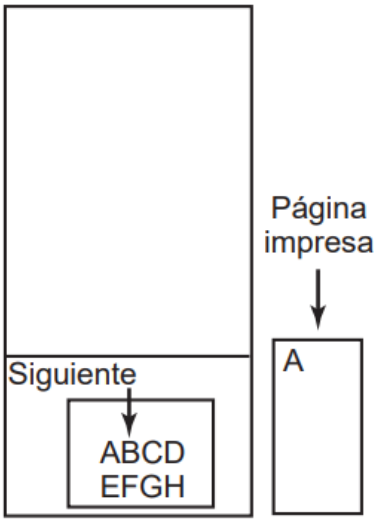


Después el proceso de usuario adquiere la impresora para escribir, haciendo una llamada al sistema para abrirla. Si la impresora está actualmente siendo utilizada por otro proceso, esta llamada fallará y devolverá un código de error o se bloqueará hasta que la impresora esté disponible, dependiendo del sistema operativo y los parámetros de la llamada. Una vez que obtiene la impresora, el proceso de usuario hace una llamada al sistema para indicar al sistema operativo que imprima la cadena en la impresora.

Después, el sistema operativo por lo general copia el búfer con la cadena a un arreglo, por ejemplo, *p* en espacio de kernel, donde se puede utilizar con más facilidad (debido a que el kernel tal vez tenga que modificar el mapa de memoria para tener acceso al espacio de usuario).

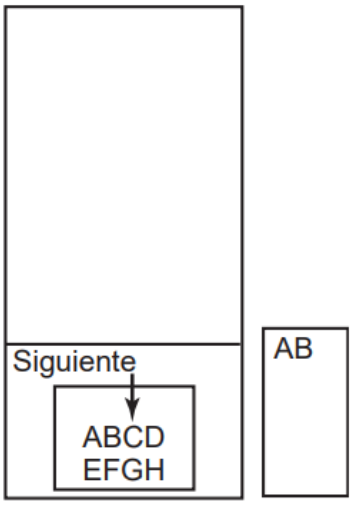
Después comprueba si la impresora está disponible en ese momento. Si no lo está, espera hasta que lo esté. Tan pronto como la impresora está disponible, el sistema operativo copia el primer carácter al registro de datos de la impresora, en este ejemplo mediante el uso de E/S por asignación de memoria. Esta acción activa la impresora. El carácter tal vez no aparezca todavía, debido a que algunas impresoras colocan en búfer una línea o una página antes de imprimir algo.

No obstante, en la figura siguiente podemos ver que se ha impreso el primer carácter y que el sistema ha marcado a “B” como el siguiente carácter a imprimir.



Tan pronto como copia el primer carácter a la impresora, el sistema operativo comprueba si la impresora está lista para aceptar otro. En general la impresora tiene un segundo registro, que proporciona su estado. El acto de escribir en el registro de datos hace que el estado se convierta en “no está lista”. Cuando el controlador de la impresora ha procesado el carácter actual, indica su disponibilidad estableciendo cierto bit en su registro de estado, o colocando algún valor en él.

En este punto el sistema operativo espera a que la impresora vuelva a estar lista. Cuando eso ocurre, imprime el siguiente carácter, como se muestra en la figura siguiente.



Este ciclo continúa hasta que se ha impreso toda la cadena. Después el control regresa al proceso de usuario.

Las acciones realizadas por el sistema operativo se sintetizan en el siguiente código.

```
copiar_del_usuario(bufer, p, cuenta);                    /* p es el búfer del kernel */
for (i=0; i<cuenta; i++) {                               /* itera en cada carácter */
        while (*reg_estado_impresora != READY);          /* itera hasta que esté lista */
        *registro_datos_impresora = p[i];                /* imprime un carácter */
}
regresar_al_usuario();
```

Primero se copian los datos en el kernel. Después el sistema operativo entra a un ciclo estrecho, imprimiendo los caracteres uno a la vez. El aspecto esencial de la E/S programada, que se ilustra con claridad en esta figura, es que después de imprimir un carácter, la CPU sondea en forma continua el dispositivo para ver si está listo para aceptar otro. Este comportamiento se conoce comúnmente ***sondeo***, ocupado en espera o ***espera activa***.

```
while (dispositivo_no_listo) {
    // la CPU revisa continuamente
}
```

La E/S programada es simple, pero tiene la desventaja de ocupar la CPU tiempo completo hasta que se completen todas las operaciones de E/S. Si el tiempo para “imprimir” un carácter es muy corto (debido a que todo lo que hace la impresora es copiar el nuevo carácter a un búfer interno), entonces está bien usar espera activa.

Ventaja	**Muy simple:** Es facilísimo de programar y no requiere chips adicionales.
Desventaja	**Invalorable desperdicio:** Mantiene a la CPU ocupada al 100% en un lazo vacío, sin poder hacer otros trabajos.
¿Cuándo sirve?	• En **sistemas embebidos** (donde la CPU no tiene ninguna otra tarea que hacer).

Además, en un sistema incrustado o embebido, donde la CPU no tiene nada más que hacer, este sondeo es razonable.

Sin embargo, en sistemas más complejos en donde la CPU tiene otros trabajos que realizar, la espera activa es ineficiente. Se necesita un mejor método de E/S.

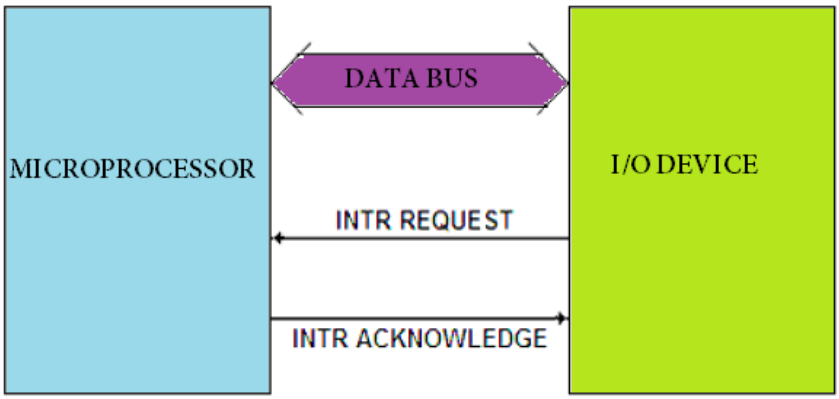
Característica	Descripción
Control	Totalmente a cargo de la CPU.
Simplicidad	Es el método más simple de E/S.
Uso de CPU	Muy ineficiente: la CPU se queda "esperando".

E/S controlada por interrupciones.

Vamos a considerar el caso de imprimir en una impresora que no coloca los caracteres en un búfer, sino que imprime cada uno a medida que va llegando. Si la impresora puede imprimir (por ejemplo,) 100 caracteres/seg, cada carácter requiere 10 mseg para imprimirse. Esto significa que después de escribir cada carácter en el registro de datos de la impresora, la CPU estará en un ciclo de inactividad durante 10 mseg, esperando a que se le permita imprimir el siguiente carácter. Este tiempo es más que suficiente para realizar un cambio de contexto y ejecutar algún otro proceso durante los 10 mseg que, de otra manera, se desperdiciarían.

La forma de permitir que la CPU haga algo más mientras espera a que la impresora esté lista es utilizar ***interrupciones***.

La ***E/S controlada por interrupciones*** es un método en el que los dispositivos de E/S avisan a la CPU cuando están listos para transferir datos o cuando una operación ha terminado, generando una interrupción. Esto evita que la CPU tenga que estar esperando activamente.



- **MICROPROCESSOR (microprocesador)**: es la **CPU**, que ejecuta programas.
- **I/O DEVICE (dispositivo de E/S)**: puede ser un teclado, disco, tarjeta de red, etc.
- **DATA BUS (bus de datos)**: canal físico por el que viajan los datos entre la CPU y el dispositivo.
- **INTR REQUEST (solicitud de interrupción)**: señal que el dispositivo envía a la CPU cuando necesita atención.
- **INTR ACKNOWLEDGE (reconocimiento de interrupción)**: señal de la CPU al dispositivo indicando que atenderá la solicitud.

Cuando se realiza la llamada al sistema para imprimir una cadena, el búfer se copia en espacio de kernel (como vimos antes) y el primer carácter se copia a la impresora, tan pronto como esté dispuesta para aceptar un carácter. En ese momento, la CPU llama al planificador y se ejecuta algún otro proceso.

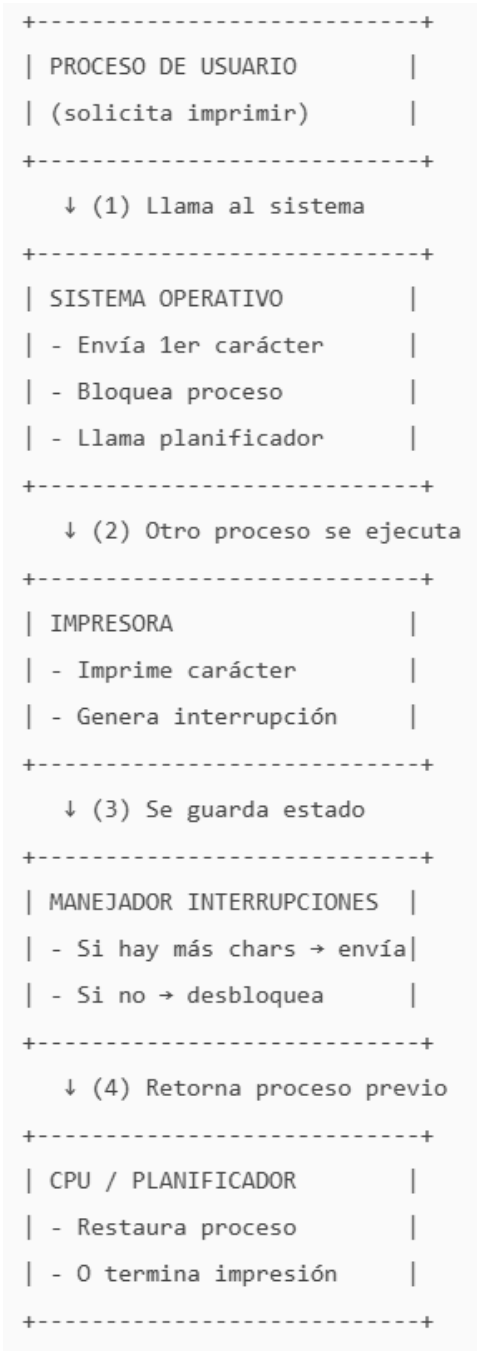
El proceso que pidió imprimir la cadena se bloquea hasta que se haya impreso toda la cadena. El trabajo realizado en la llamada al sistema se muestra en el código siguiente.

```
copiar_del_usuario(bufer, p, cuenta);
habilitar_interrupciones();
while (*reg_estado_impresora != READY);
*registro_datos_impresora = p[0];
planificador();
```

Cuando la impresora ha impreso el carácter, y está preparada para aceptar el siguiente, genera una interrupción. Esta interrupción detiene el proceso actual y guarda su estado. Después se ejecuta el procedimiento de servicio de interrupciones de la impresora. Una versión cruda de este código se muestra en el siguiente código.

```
if (cuenta==0) {
   desbloquear_usuario();
} else {
   *registro_datos_impresora = p[i];
   cuenta=cuenta – 1;
   i = i + 1;
}
reconocer_interrupcion();
regresar_de_interrupcion();
```

Si no hay más caracteres por imprimir, el manejador de interrupciones realiza una acción para desbloquear al proceso de usuario. En caso contrario, imprime el siguiente carácter, reconoce la interrupción y regresa al proceso que se estaba ejecutando justo antes de la interrupción, que continúa desde donde se quedó.



E/S mediante el uso de DMA.

Una obvia desventaja de la E/S controlada por interrupciones es que ocurre una interrupción en cada carácter. Las interrupciones requieren tiempo, por lo que este esquema desperdicia cierta cantidad de tiempo de la CPU. Es decir, se introduce sobrecarga (***overhead***) por interrupción.

Una solución es utilizar DMA. Acá la idea es permitir que el controlador de DMA alimente los caracteres a la impresora uno a la vez, sin que la CPU se moleste.

En esencia, el DMA es muy similar a la E/S programada, sólo que el controlador de DMA realiza todo el trabajo en vez de la CPU principal. Esta estrategia requiere hardware especial (el controlador de DMA) pero libera la CPU durante la E/S para realizar otro trabajo. En la figura siguiente se muestra un esquema del código.

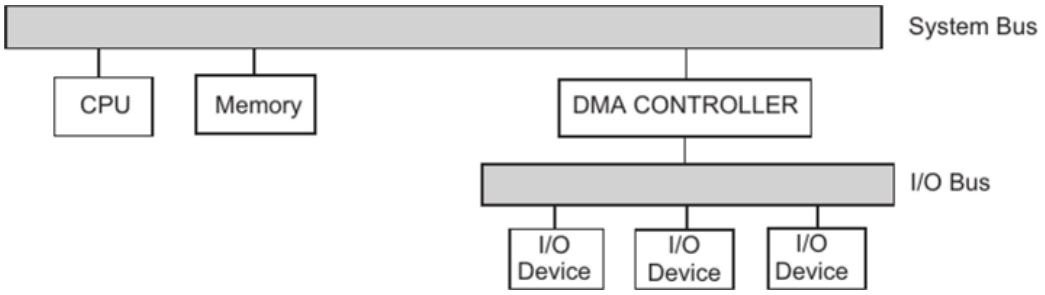
```
copiar_del_usuario(bufer, p, cuenta);
establecer_controlador_DMA();
planificador();
```

```
reconocer_interrupcion();
desbloquear_usuario();
regresar_de_interrupcion();
```

La gran ganancia con DMA es reducir el número de interrupciones, de una por cada carácter a una por cada búfer impreso. Si hay muchos caracteres y las interrupciones son lentas, esto puede ser una gran mejora.

SIN DMA	CON DMA
- CPU mueve datos directamente	- Controlador DMA mueve datos
- Interrupción por cada unidad transferida	
- Alta carga para la CPU	- CPU queda libre para otras tareas
- Transferencia rápida (CPU)	- Transferencia más lenta (DMA)
- Mayor número de cambios de contexto	- Menos cambios de contexto
- CPU controla todo el proceso	- Control delegado al DMA

Por otra parte, el controlador de DMA es comúnmente más lento que la CPU principal. Si el controlador de DMA no puede controlar el dispositivo a toda su velocidad, o si la CPU por lo general no tiene nada que hacer mientras espera la interrupción de DMA, entonces puede ser mejor utilizar la E/S controlada por interrupción o incluso la E/S programada. De todas formas, la mayor parte del tiempo vale la pena usar DMA.



Los sistemas operativos modernos como Linux, al igual que en el caso del direccionamiento, suelen utilizar un esquema híbrido y dinámico para la transferencia de datos. No se quedan con un solo método, sino que aplican el mecanismo más eficiente según el tipo de hardware, la velocidad del dispositivo y la cantidad de datos que se deban mover.

Método	¿Cuándo lo usa Linux?	¿Cómo actúa la CPU?
Por DMA	Con dispositivos **rápidos** (discos NVMe o placas de video).	Da la orden y el chip DMA se encarga de transferir todo el bloque de datos.
Por Interrupciones	Con dispositivos **lentos** (teclado, mouse o audio).	Ejecuta otros procesos y regresa solo cuando el hardware le envía un aviso.
Por Espera Activa	Solo en casos especiales de **velocidad extrema**.	Consulta continuamente el estado del dispositivo hasta que este responde.

Referencias y bibliografía de la sección.

- Tanenbaum 5-1 / 5-2.
- Silberschatz 13-1.

Capas principales del software de E/S.

Por lo general, el software de E/S se organiza en cuatro capas, como se muestra en la siguiente figura.

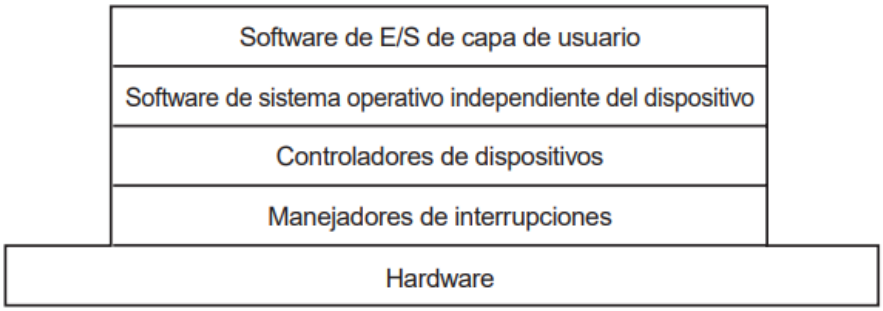


Ilustración 12 -Capas del sistema de software de E/S.

Cada capa tiene una función bien definida que realizar, y una interfaz bien definida para los niveles adyacentes. La funcionalidad y las interfaces difieren de un sistema a otro.

Manejadores de interrupciones.

Aunque la E/S programada es útil algunas veces, para la mayor parte de las operaciones de E/S las interrupciones son un hecho incómodo de la vida y no se pueden evitar. Deben ocultarse en la profundidad de las entrañas del sistema operativo, de manera que éste sepa lo menos posible de ellas. La mejor manera de ocultarlas es hacer que el ***driver*** o controlador (software) que inicia una operación de E/S se bloquee hasta que se haya completado la E/S y ocurra la interrupción.

```
[Aplicación]
    |
    | 1. Pide un dato
    ▼
[Sistema Operativo] ——— (Incluye los DRIVERS que
    |                    manejan las interrupciones)
    ▼
[Hardware]
```

El driver se puede bloquear a sí mismo realizando una llamada a ***down*** en un semáforo, una llamada a ***wait*** en una variable de condición, una llamada a ***receive*** en un mensaje o algo similar, por ejemplo.

Cuando ocurre la interrupción, el procedimiento de interrupciones hace todo lo necesario para poder manejarla. Después puede desbloquear el driver que la inició.

En algunos casos sólo completará up en un semáforo. En otros casos realizará una llamada a ***signal*** en una variable de condición en un monitor. En otros más enviará un mensaje al controlador bloqueado. En todos los casos, el efecto neto de la interrupción será que un controlador que estaba bloqueado podrá ejecutarse ahora.

Este modelo funciona mejor si los drivers están estructurados como procesos del kernel, con sus propios estados, pilas y contadores del programa.

1. El **driver (SW)** le da la orden de trabajo al chip del dispositivo.
2. El **driver (SW)** se bloquea a sí mismo en la memoria principal usando un semáforo para no gastar CPU.
3. El **controlador físico (HW)** hace el trabajo en paralelo.
4. El **controlador físico (HW)** envía una interrupción al terminar, lo que despierta al **driver (SW)** para que continúe su ejecución.

Desde luego que en realidad esto no es tan simple. Procesar una interrupción no es cuestión de sólo tomar la interrupción, llamar a ***up*** en algún semáforo y después ejecutar una instrucción IRET[4] para regresar de la interrupción al proceso anterior. Hay mucho más trabajo involucrado para el sistema operativo.

Veamos un esquema de este trabajo como una serie de pasos que se deben llevar a cabo en el software, una vez que se haya completado la interrupción de hardware. Hay que recalcar que los detalles dependen mucho del sistema, por lo que algunos de los pasos que se listan a continuación tal vez no sean necesarios en una máquina específica, y tal vez se requieran otros que no estén listados. Además, los pasos que se llevan a cabo pueden estar en distinto orden en algunas máquinas.

1. Guardar los registros (incluyendo el PSW[5]) que no han sido guardados por el hardware de la interrupción.
2. Establecer un contexto para el procedimiento de servicio de interrupciones. Para ello tal vez sea necesario establecer el TLB[6], la MMU[7] y una tabla de páginas.
3. Establecer una pila para el procedimiento de servicio de interrupciones.
4. Reconocer el controlador de interrupciones. Si no hay un controlador de interrupciones centralizado, rehabilitar las interrupciones.
5. Copiar los registros desde donde se guardaron (posiblemente en alguna pila) a la tabla de procesos.
6. Ejecutar el procedimiento de servicio de interrupciones. Éste extraerá información de los registros del controlador de dispositivos que provocó la interrupción.

[4] La instrucción IRET (Interrupt Return) en la arquitectura x86 y x86-64 es una instrucción de ensamblaje que se utiliza para regresar de un procedimiento de interrupción al programa que fue interrumpido.
[5] El registro PSW (Program Status Word) es un registro de estado del programa. Es un área de memoria o registro que contiene información sobre el estado de un programa que está siendo ejecutado por el sistema operativo. Este registro incluye, entre otras cosas, indicadores de errores, códigos de condición y la dirección de la siguiente instrucción a ejecutar.
[6] El Búfer de Traducción Lookaside (TLB, por sus siglas en inglés) es una caché de memoria que acelera la traducción de direcciones virtuales a direcciones físicas en sistemas operativos con memoria virtual. Es una pieza clave de la Unidad de Gestión de Memoria (MMU) que permite un acceso más rápido a la memoria.
[7] La MMU (Memory Management Unit o Unidad de Gestión de Memoria) es un componente de hardware que traduce direcciones virtuales a direcciones físicas. Esta traducción permite a los procesos utilizar la memoria de forma segura y eficiente, incluso si no hay suficiente memoria física disponible.

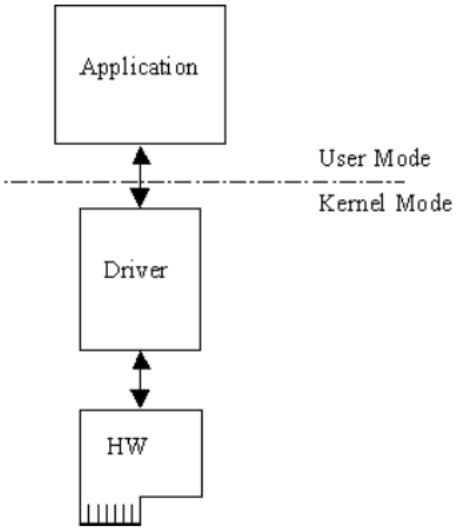
7. Elegir cuál proceso ejecutar a continuación. Si la interrupción ha ocasionado que cierto proceso de alta prioridad que estaba bloqueado cambie al estado listo, puede elegirse para ejecutarlo en ese momento.
8. Establecer el contexto de la MMU para el proceso que se va a ejecutar a continuación. También puede ser necesario establecer un TLB.
9. Cargar los registros del nuevo proceso, incluyendo su PSW.
10. Empezar a ejecutar el nuevo proceso.

Como se puede ver, el procesamiento de interrupciones no carece de importancia. También ocupa un considerable número de instrucciones de la CPU, en especial en máquinas en las que hay memoria virtual y es necesario establecer tablas de páginas, o se tiene que guardar el estado de la MMU (por ejemplo, los bits *R* y *M*). En algunas máquinas, el TLB y la caché de la CPU tal vez también tengan que manejarse al cambiar entre los modos de usuario y de kernel, lo cual requiere ciclos de máquina adicionales.

1. Controlador Físico	**Hardware** (Chips)	Hace el trabajo físico y envía la señal eléctrica (interrupción).
2. Manejador de Interrupciones	**Software** (Código)	Atiende la señal eléctrica de emergencia y desbloquea al driver.
3. Driver	**Software** (Código)	Estaba bloqueado; se activa y continúa con la ejecución de su código.

Drivers de dispositivos.

Cada ***controlador de dispositivo*** (hardware) tiene ciertos registros de dispositivos (que son celdas de memoria físicas dentro del chip) que se utilizan para darle comandos o ciertos registros de dispositivos que se utilizan para leer su estado, o ambos.



El número de registros de dispositivos y la naturaleza de los comandos varían radicalmente de un dispositivo a otro. Por ejemplo, un controlador de mouse tiene que aceptar información del mouse que le indica qué tanto se ha desplazado y cuáles botones están oprimidos en un momento dado. Por el contrario, un controlador de disco tal vez tenga que saber todo acerca de los sectores, pistas, cilindros, cabezas, movimiento del brazo, los propulsores del motor, los tiempos de asentamiento de las cabezas y todos los demás mecanismos para hacer que el disco funcione en forma apropiada. Obviamente, cada dispositivo utilizará drivers muy distintos.

Registro de Comando	El **Driver (SW)** escribe aquí para ordenar una acción.
Registro de Estado	El **Controlador (HW)** escribe aquí los cambios físicos; el Driver (SW) los lee.

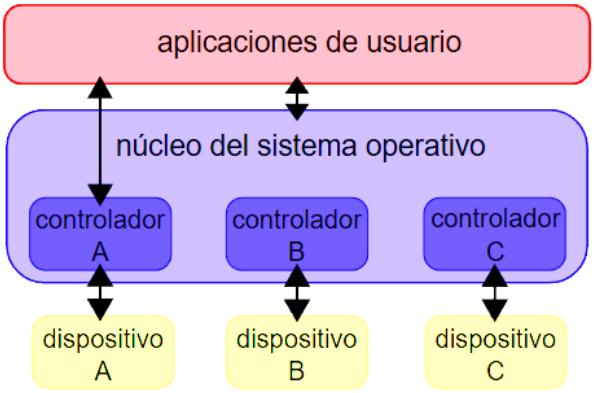
Como consecuencia, cada dispositivo de E/S conectado a una computadora necesita cierto código específico para controlarlo. Este código, conocido como ***driver***, es escrito por el fabricante del dispositivo y se incluye junto con el mismo. Como cada sistema operativo necesita sus propios drivers, los fabricantes de dispositivos por lo común los proporcionan para varios sistemas operativos populares.

Componente	¿Qué es?	¿Quién lo fabrica?
Driver	**Software** (Código)	El fabricante del dispositivo (para cada sistema operativo).
Controlador	**Hardware** (Chips)	El fabricante del circuito electrónico.

Cada driver maneja un tipo de dispositivo o, a lo más, una clase de dispositivos estrechamente relacionados. Por ejemplo, un driver de disco SATA puede manejar por lo general varios discos SATA de distintos tamaños y velocidades. Por otro lado, un mouse y una palanca de mandos son tan distintos que por lo general se requieren drivers diferentes.

Sin embargo, no hay una restricción técnica en cuanto a que un driver controle varios dispositivos no relacionados. Simplemente no es una buena idea.

Para poder utilizar el hardware del dispositivo (es decir, los registros del controlador físico), el driver por lo general tiene que formar parte del kernel del sistema operativo.



Sin embargo, en la realidad es posible construir drivers que se ejecuten en el espacio de usuario, con llamadas al sistema para leer y escribir en los registros del dispositivo. Este diseño aísla al kernel de los drivers, y a un driver de otro, eliminando una fuente importante de fallas en el sistema: drivers con errores que interfieren con el kernel de una manera u otra.

Para construir sistemas altamente confiables, ésta es, en definitiva, la mejor manera de hacerlo. Un ejemplo de un sistema donde los drivers de dispositivos se ejecutan como procesos de usuario es ***MINIX 3***. Sin embargo, como la mayoría de los demás sistemas operativos de escritorio esperan que los drivers se ejecuten en el kernel, éste es el modelo que consideraremos.

En un sistema de arquitectura ***monolítica*** pura (como el diseño clásico de Linux), todo el código del driver corre dentro del espacio del kernel.

Sin embargo, para organizar ese código y no ralentizar todo el sistema, los kernels monolíticos modernos dividen la ejecución del driver en dos etapas temporales (pero ambas ocurren dentro del kernel).

- **La Parte Alta (Top Half):** Es la función que se activa de inmediato cuando el hardware envía una señal eléctrica (interrupción). Hace lo mínimo indispensable (como guardar datos rápidos en un búfer de la memoria principal) y desbloquea el flujo del driver.
- **La Parte Baja (Bottom Half):** Es el código que procesa las tareas más pesadas un instante después, de manera ordenada, pero siempre operando dentro del espacio seguro y privilegiado del kernel.

Aunque hoy en día sistemas como Linux permiten que ciertos drivers específicos de tipo experimental o de sistemas de archivos corran en el espacio de usuario mediante herramientas como FUSE o UIO, la regla general de la arquitectura monolítica es que los drivers de dispositivos corren íntegramente en el kernel.

En ***Windows***, sin embargo, el escenario es diferente y más complejo porque utiliza una arquitectura ***híbrida***. Una parte corre en el kernel y otra parte puede correr en el espacio de usuario.

Como los diseñadores de cada sistema operativo no saben qué piezas de código (drivers) escritas por terceros se instalarán en él, necesita tener una arquitectura que permita dicha instalación. Esto implica tener un modelo bien definido de lo que hace un driver y la forma en que interactúa con el resto del sistema operativo. Por lo general, los drivers de dispositivos se posicionan debajo del resto del sistema operativo, como se ilustra en la figura siguiente.

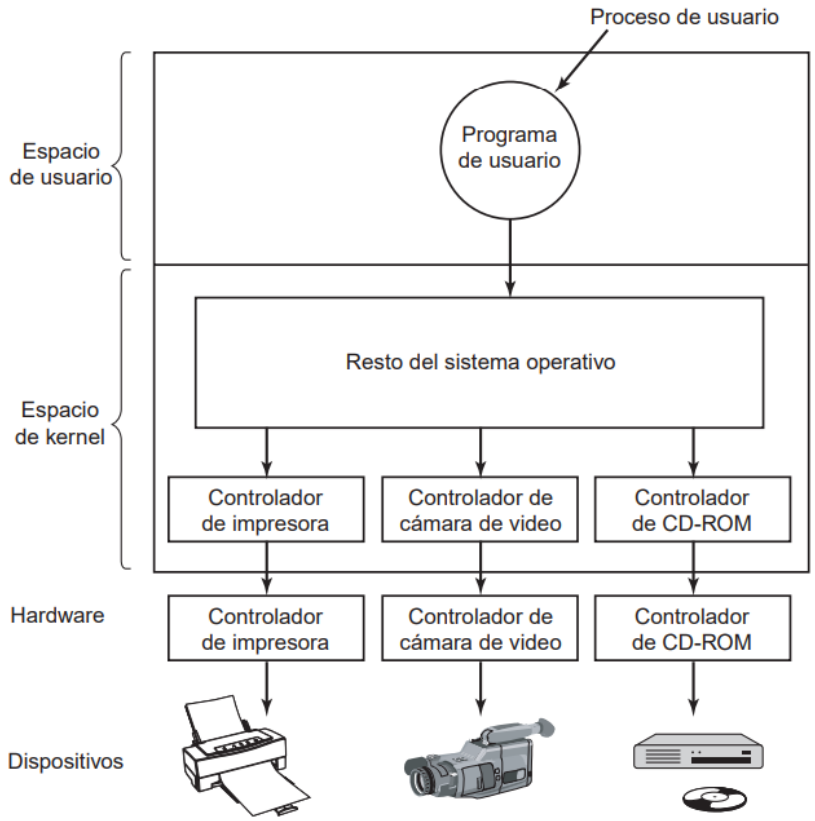
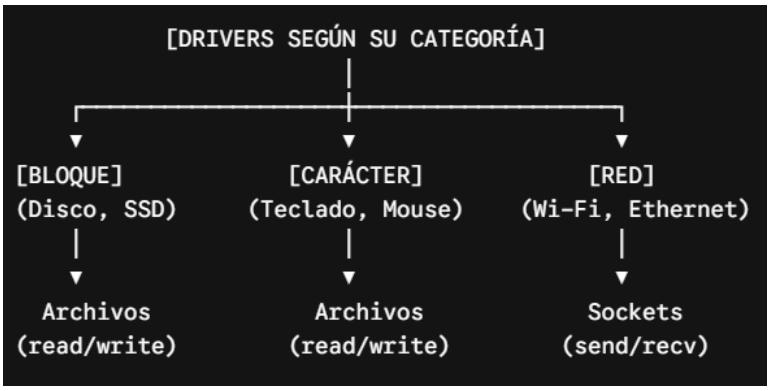


Ilustración 13 - Posicionamiento lógico del software controlador de dispositivos. En realidad, toda la comunicación entre el software controlador y los controladores de dispositivos pasa a través del bus.

Generalmente los sistemas operativos clasifican los drivers en una de un pequeño número de categorías. Las categorías más comunes son los ***drivers de bloque*** como los drivers de discos, que contienen varios bloques de datos que se pueden direccionar de manera independiente, y los ***drivers de carácter*** como los drivers de teclados y las impresoras, que generan o aceptan un flujo de caracteres.

Aunque la división bloque vs. carácter cubre la gran mayoría del hardware tradicional, los sistemas operativos modernos (como Linux) suelen añadir una tercera categoría principal: dispositivos de red.

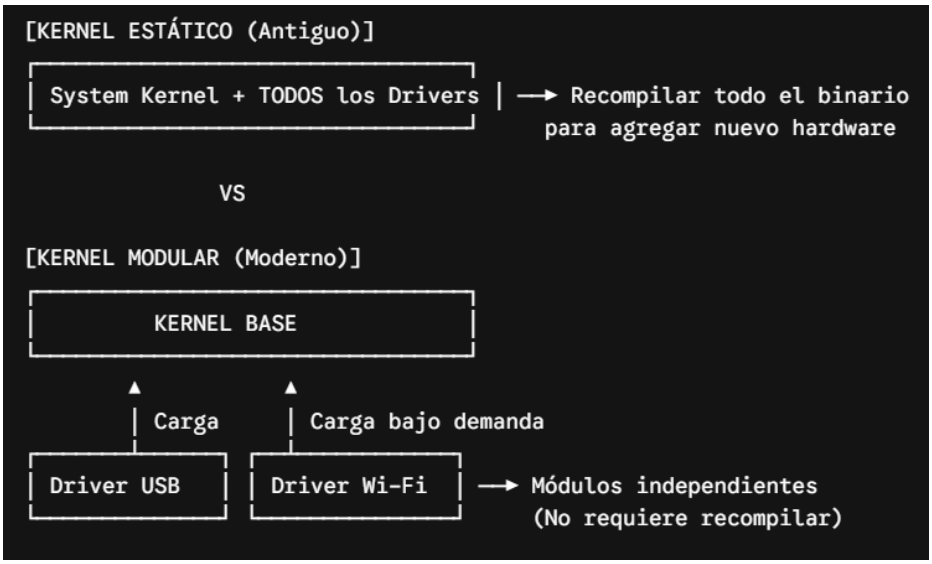
Estos dispositivos de red no encajan bien en las otras dos porque no se manejan mediante el sistema de archivos estándar (no se leen con read() o write() sobre un archivo en /dev), sino a través de la interfaz de sockets, orientada al envío y recepción de paquetes discretos de datos (tramas/datagramas).



La mayoría de los sistemas operativos definen una interfaz estándar que todos los drivers deben aceptar. Estas interfaces consisten en varios procedimientos que el resto del sistema operativo puede llamar para hacer que el driver realice un trabajo para él. Los procedimientos ordinarios son los que se utilizan para leer un bloque (dispositivo de bloque) o escribir una cadena de caracteres (dispositivo de carácter).

Concepto	¿Qué hace?	En lenguaje cotidiano...
Abstracción	**Oculta la complejidad** física del hardware.	Hace que el sistema operativo vea a cualquier disco simplemente como una lista de bloques numerados.
Interfaz Uniforme	**Unifica las órdenes** mediante funciones fijas (`read` , `write`).	Obliga a todos los drivers a aceptar los mismos comandos estándar, sin importar la marca del dispositivo.

En algunos casos, el sistema operativo es un solo programa binario que contiene compilados en él todos los drivers que requerirá. Este esquema fue la norma durante años con los sistemas UNIX, debido a que eran ejecutados por centros de computadoras y los dispositivos de E/S cambiaban pocas veces. Si se agregaba un nuevo dispositivo, el administrador del sistema simplemente recompilaba el kernel con el nuevo driver para crear un nuevo binario.



Con la llegada de las computadoras personales y la multitud de dispositivos de E/S, este modelo ya no era funcional. Pocos usuarios son capaces de volver a compilar o vincular el kernel, aun si tienen el código fuente o los módulos de código objeto, que no siempre es el caso. En vez de ello, los sistemas operativos (empezando con MS-DOS) se inclinaron por un modelo en el que el driver se carga en forma dinámica en el sistema durante la ejecución. Los diferentes sistemas manejan la carga de drivers en distintas formas.

Antiguo (Kernel Estático)	Recompilando todo el núcleo para crear un único binario nuevo.	Administradores de sistemas e ingenieros.
Moderno (Kernel Modular)	**Se añade el driver de forma dinámica sin recompilar** el kernel.	El propio sistema operativo de forma automática.

Un driver de dispositivo tiene varias funciones. La más obvia es aceptar peticiones abstractas de lectura y escritura del ***software independiente del dispositivo*** que está por encima de él, y ver que se lleven a cabo. Pero también hay otras tantas funciones que deben realizar. Por ejemplo, el driver debe inicializar el dispositivo, si es necesario. También puede tener que administrar sus propios requerimientos y eventos del registro.

Muchos tipos de drivers de dispositivos tienen una estructura general similar. Un driver ordinario empieza por comprobar los parámetros de entrada para ver si son válidos. De no ser así se devuelve un error. Si son válidos tal vez sea necesaria una traducción de términos abstractos a concretos. Para un driver de disco, esto puede significar tener que convertir un número de bloque lineal en los números de cabeza, pista, sector y cilindro para la geometría del disco.

A continuación, el driver puede comprobar si el dispositivo se encuentra en uso. De ser así, la petición se pondrá en cola para procesarla después. Si el dispositivo está inactivo, el estado del hardware se examinará para ver si la petición se puede manejar en ese momento. Tal vez sea necesario encender el dispositivo o iniciar un motor para que puedan empezar las transferencias. Una vez que el dispositivo esté encendido y listo para trabajar, puede empezar el control en sí.

Controlar el dispositivo significa enviarle una secuencia de comandos. El driver es el lugar en donde se determina la secuencia de comandos, dependiendo de lo que se deba hacer. Una vez que el driver sabe qué comandos va a emitir, empieza a escribirlos en los registros de dispositivo del controlador.

Después de escribir cada comando para el controlador, tal vez sea necesario comprobar si el controlador aceptó el comando y está preparado para aceptar el siguiente. Esta secuencia continúa hasta que se hayan emitido todos los comandos.

Algunos drivers pueden recibir una lista de comandos (en memoria) y se les puede indicar que los procesen todos por sí mismos, sin necesidad de ayuda del sistema operativo.

Una vez que se han emitido los comandos, se dará una de dos situaciones. En muchos casos el driver de dispositivos debe esperar hasta que el controlador realice cierto trabajo para él, por lo que se bloquea a sí mismo hasta que llegue la interrupción para desbloquearlo. Sin embargo, en otros casos la operación termina sin retraso, por lo que el driver no necesita bloquearse. Como ejemplo de esta última situación, para desplazar la pantalla en modo de carácter se requiere escribir sólo unos cuantos bytes en los registros del controlador. No es necesario ningún tipo de movimiento mecánico, por lo que toda la operación se puede completar en nanosegundos.

En el primer caso, el driver bloqueado será despertado por la interrupción. En el segundo caso nunca pasará al estado inactivo. De cualquier forma, una vez que se ha completado la operación, el driver debe comprobar si hay errores. Si todo está bien, el driver puede hacer que pasen datos al software independiente del dispositivo (por ejemplo, un bloque que se acaba de leer).

Por último, devuelve cierta información de estado para reportar los errores de vuelta al que lo llamó.

Si hay otras peticiones en la cola, ahora se puede seleccionar e iniciar una de ellas. Si no hay nada en la cola, el driver se bloquea en espera de la siguiente petición.

Este modelo simple es sólo una aproximación a la realidad. Muchos factores hacen el código mucho más complicado. Por una parte, un dispositivo de E/S puede completar su operación mientras haya un driver en ejecución, con lo cual se interrumpe el driver. La interrupción puede hacer que se ejecute un driver de dispositivo. De hecho, puede hacer que el driver actual se ejecute. Por ejemplo, mientras el controlador de red está procesando un paquete entrante, puede llegar otro paquete. En consecuencia, un driver tiene que ser ***reentrante***, lo cual significa que un driver en ejecución tiene que esperar a ser llamado una segunda vez antes de que se haya completado la primera llamada.

En un sistema con “conexión en caliente”[8] es posible agregar o eliminar dispositivos mientras la computadora está en ejecución. Como resultado, mientras un driver está ocupado leyendo de algún dispositivo, el sistema puede informarle que el usuario ha quitado de manera repentina ese dispositivo del sistema. No sólo se debe abortar la transferencia actual de E/S sin dañar ninguna estructura de datos del kernel, sino que cualquier petición pendiente del ahora desaparecido dispositivo debe eliminarse también, con cuidado, del sistema, avisando a los que hicieron la llamada. Además, la adición inesperada de nuevos dispositivos puede hacer que el kernel haga malabares con los recursos (por ejemplo, las líneas de petición de interrupciones), quitando los anteriores al driver y dándole nuevos recursos en vez de los otros.

El driver no puede hacer llamadas al sistema, pero a menudo necesita interactuar con el resto del kernel. Por lo general se permiten llamadas a ciertos procedimientos del kernel. Por ejemplo, comúnmente hay llamadas para asignar y desasignar páginas fijas de memoria para usarlas como búferes. Otras llamadas útiles se necesitan para administrar la MMU, los temporizadores, el controlador de DMA, el controlador de interrupciones, etcétera.

Software de E/S independiente del dispositivo.

Aunque parte del software de E/S es específico para cada dispositivo, otras partes de este son independientes de los dispositivos. El límite exacto entre los drivers y el software independiente del dispositivo depende del sistema (y del dispositivo), debido a que ciertas funciones que podrían realizarse de una manera independiente al dispositivo pueden realizarse en los drivers, por eficiencia u otras razones. Las funciones que se muestran en la figura siguiente se realizan comúnmente en el software independiente del dispositivo.

Interfaz uniforme para controladores de dispositivos
Uso de búfer
Reporte de errores
Asignar y liberar dispositivos dedicados
Proporcionar un tamaño de bloque independiente del dispositivo

Ilustración 14 - Funciones del software de E/S independiente del dispositivo.

La función básica del software independiente del dispositivo es realizar las funciones de E/S que son comunes para todos los dispositivos y proveer una interfaz uniforme para el software a nivel de usuario.

[8] "Hotswap" o "cambio en caliente" se refiere a la capacidad de un componente informático para ser instalado o sustituido sin necesidad de apagar o detener la computadora. Esto significa que el componente puede conectarse y desconectarse sin interrumpir la operación del sistema.

Interfaz uniforme para los controladores de software de dispositivos.

Una importante cuestión en un sistema operativo es cómo hacer que todos los dispositivos de E/S y sus drivers se vean más o menos iguales. Si los discos, las impresoras, los teclados, etc., se conectan de distintas maneras, cada vez que llegue un nuevo dispositivo el sistema operativo deberá modificarse para éste. No es conveniente tener que modificar el sistema operativo para cada nuevo dispositivo.

Un aspecto de esta cuestión es la interfaz entre los drivers de dispositivos y el resto del sistema operativo. En la figura siguiente se ilustra una situación en la que cada driver de dispositivo tiene una interfaz distinta con el sistema operativo.

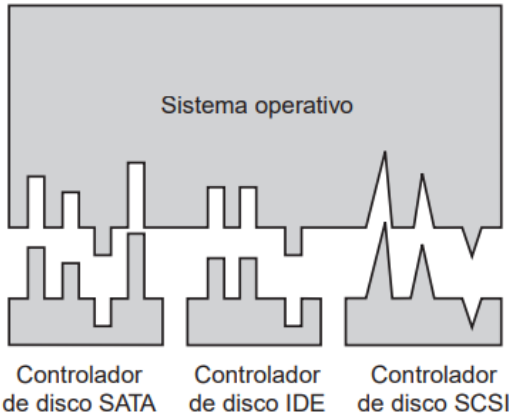


Ilustración 15 - Sin una interfaz de controlador estándar.

Lo que esto significa es que las funciones de driver disponibles para que el sistema pueda llamarlas difieren de un driver a otro. Además, podría significar que las funciones de kernel que el driver necesita también difieren de driver en driver. En conjunto, quiere decir que la interfaz para cada nuevo driver requiere mucho esfuerzo de programación.

Por el contrario, en la figura que tenemos a continuación podemos ver un diseño distinto, en el que todos los drivers tienen la misma interfaz.

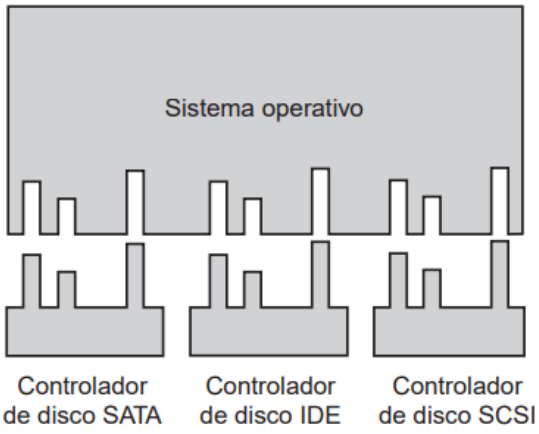


Ilustración 16 - Con una interfaz de controlador estándar.

Ahora es mucho más fácil conectar un nuevo driver, siempre y cuando sea compatible con su interfaz. Esto también significa que los escritores de los drivers saben lo que se espera de ellos. En la práctica no todos los dispositivos son absolutamente idénticos, pero por lo general hay sólo un pequeño número de tipos de dispositivos, e incluso éstos en general son casi iguales.

La manera en que funciona es la siguiente. Para cada clase de dispositivos, como los discos o las impresoras, el sistema operativo define un conjunto de funciones que el driver debe proporcionar.

Para un disco, estas funciones incluyen naturalmente la lectura y la escritura, pero también encender y apagar la unidad, aplicar formato y otras cosas relacionadas con los discos. A menudo, el driver contiene una tabla con apuntadores a sí mismo para estas funciones. Cuando se carga el driver, el sistema operativo registra la dirección de esta tabla de apuntadores a funciones, por lo que cuando necesita llamar a una de las funciones, puede hacer una llamada indirecta a través de esta tabla. Esta tabla de apuntadores a funciones define la interfaz entre el driver y el resto del sistema operativo. Todos los dispositivos de una clase dada (discos, impresoras, etc.) deben obedecerla.

Otro aspecto de tener una interfaz uniforme es la forma en que se nombran los dispositivos. El software independiente del dispositivo se hace cargo de asignar nombres de dispositivo simbólicos al driver apropiado. Por ejemplo, en UNIX el nombre de un dispositivo como */dev/disk0* especifica de manera única el nodo-i para un archivo especial, y este nodo-i contiene el ***número mayor de dispositivo***, que se utiliza para localizar el driver apropiado. El nodo-i también contiene el ***número menor de dispositivo***, que se pasa como un parámetro al driver para poder especificar la unidad que se va a leer o escribir. Todos los dispositivos tienen números mayores y menores, y para acceder a todos los drivers se utiliza el número mayor de dispositivo para seleccionar el driver.

La protección está muy relacionada con la denominación. El sistema debe evitar que los usuarios accedan a dispositivos que tienen prohibido utilizar. Tanto en UNIX como en Windows, los dispositivos aparecen en el sistema de archivos como objetos con nombre, lo cual significa que las reglas de protección ordinarias para los archivos también se aplican a los dispositivos de E/S. Así, el administrador del sistema puede establecer los permisos apropiados para cada dispositivo.

Uso de búfer.

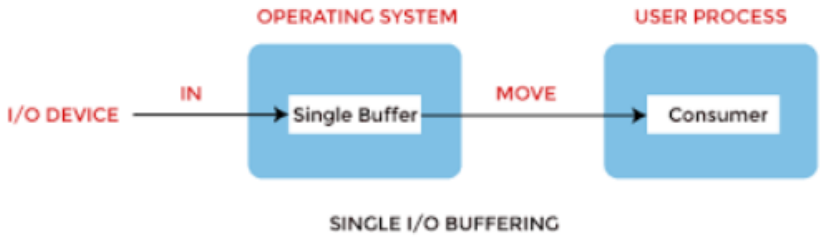
El uso de búfer es otra cuestión, tanto para los dispositivos de bloque como los de carácter, por una variedad de razones.

Un búfer es un espacio de almacenamiento temporal ubicado, muchas veces, en la memoria principal. Se utiliza para guardar datos mientras se transfieren entre un emisor y un receptor (por ejemplo, entre el disco duro y un proceso de usuario, o entre el teclado y el sistema operativo).

Existen varias razones por las cuales tanto los dispositivos de bloque como los de carácter necesitan utilizar búferes de manera indispensable.

Una de estas razones podría ser la diferencia de velocidad entre dispositivos. El procesador y la memoria principal funcionan a velocidades extremadamente altas, mientras que los dispositivos de E/S (como un disco o un teclado) son infinitamente más lentos. El búfer permite que el dispositivo lento vaya depositando sus datos poco a poco en él, evitando que la CPU pierda tiempo esperando byte por byte.

Otro motivo que justifica la implementación de un búfer es la diferencia en el tamaño de los datos. Un proceso de usuario puede querer leer datos en trozos pequeños (por ejemplo, de a 1 byte por vez), pero el dispositivo de bloque (como un disco) solo trabajo con datos en paquetes grandes (bloques de 4096 bytes). El sistema operativo lee el bloque completo del disco, lo guarda en un búfer en la memoria principal y luego le va entregando al proceso los bytes individuales a medida que los solicita.



Consideremos un proceso que desea leer datos de un módem. Una posible estrategia para lidiar con los caracteres entrantes es hacer que el proceso de usuario realice una llamada al sistema ***read*** y se bloquee en espera de un carácter. Cada carácter que llega produce una interrupción. El procedimiento de servicio de interrupciones entrega el carácter al proceso de usuario y lo desbloquea. Después de colocar el carácter en alguna parte, el proceso lee otro carácter y se bloquea de nuevo. Este modelo se indica en la parte *a* de la figura siguiente.

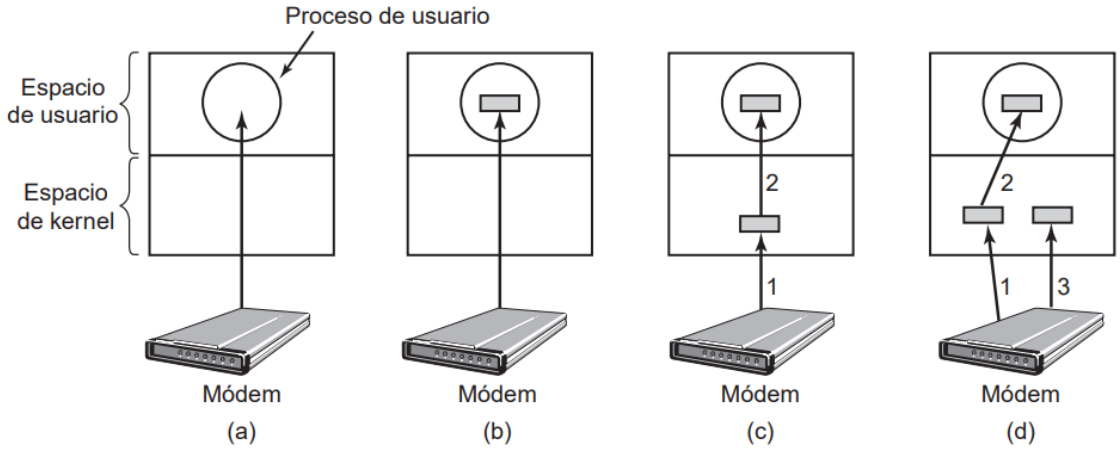


Ilustración 17 - (a) Entrada sin búfer. (b) Uso de búfer en espacio de usuario. (c) Uso de búfer en el kernel, seguido de la acción de copiar al espacio de usuario. (d) Uso de doble búfer en el kernel.

El problema con esta forma de hacer las cosas es que el proceso de usuario tiene que iniciarse por cada carácter entrante. Permitir que un proceso se ejecute muchas veces durante tiempos cortos de ejecución es ineficiente, por lo que este diseño no es bueno.

En la parte *b* de la figura se muestra una mejora. Acá el proceso de usuario proporciona un búfer de *n* caracteres en espacio de usuario y realiza una lectura de *n* caracteres. El procedimiento de servicio de interrupciones coloca los caracteres entrantes en este búfer hasta que se llena. Después despierta al proceso de usuario. Este esquema es mucho más eficiente que el anterior, pero tiene una desventaja: si el búfer se página cuando llega un carácter, el búfer se podría bloquear en la memoria; pero si muchos procesos empiezan a bloquear páginas en memoria, se reducirá la reserva de páginas disponibles y se degradará el rendimiento.

Otro método más es crear un búfer dentro del kernel y hacer que el manejador de interrupciones coloque ahí los caracteres, como se muestra en la parte *c* de la figura anterior. Cuando este búfer está lleno, se trae la página con el búfer de usuario en caso de ser necesario, y el búfer se copia ahí en una operación.

Este esquema es mucho más eficiente. Sin embargo, incluso este esquema presenta un inconveniente: los caracteres que llegan mientras se está trayendo del disco la página con el búfer de usuario, mientras el búfer está lleno, no habrá lugar en dónde ponerlos.

Una solución es tener un segundo búfer de kernel. Una vez que se llena el primero, pero antes de que se haya vaciado, se utiliza el segundo como se muestra en la parte *d* de la figura.

Cuando se llena el segundo búfer, está disponible para copiarlo al usuario (suponiendo que el usuario lo haya pedido). Mientras el segundo búfer se copia al espacio de usuario, el primero se puede utilizar para nuevos caracteres. De esta forma, los dos búferes toman turnos: mientras uno se copia al espacio de usuario, el otro está acumulando nuevos datos de entrada.

Un esquema de uso de búferes como éste se conoce como ***uso de doble búfer***.

Otra forma de uso de búfer que se utiliza ampliamente es el ***búfer circular***. Consiste en una región de memoria y dos apuntadores. Un apuntador apunta a la siguiente palabra libre, en donde se pueden colocar nuevos datos. El otro apuntador apunta a la primera palabra de datos en el búfer que todavía no se ha removido. En muchas situaciones, el hardware avanza el primer apuntador a medida que agrega nuevos datos (por ejemplo, los que acaban de llegar de la red) y el sistema operativo avanza el segundo apuntador a medida que elimina y procesa datos. Ambos apuntadores regresan a la parte final después de llegar a la parte superior.

El uso de búfer es también importante en la salida. Por ejemplo, considere cómo se envían datos al módem sin uso de búfer mediante el modelo de la parte *b* de la figura anterior. El proceso de usuario ejecuta una llamada al sistema ***write*** para imprimir *n* caracteres. El sistema tiene dos opciones en este punto. Puede bloquear el usuario hasta que se hayan escrito todos los caracteres, pero se podría requerir mucho tiempo a través de una línea telefónica lenta. También podría liberar de inmediato al usuario y realizar la operación de E/S mientras el usuario realiza unos cálculos más, pero esto produce un problema aún peor: el proceso de usuario desconoce que se ha completado la salida y puede reutilizar el búfer.

El sistema podría generar una señal o interrupción de software, pero ese estilo de programación es difícil y está propenso a condiciones de competencia. Una mucho mejor solución es que el kernel copie los datos a un búfer, de manera similar a la parte *c* de la figura

anterior (pero en sentido opuesto), y que desbloquee al que hizo la llamada de inmediato. Entonces no importa cuándo se haya completado la operación de E/S. El usuario es libre de reutilizar el búfer al instante en que se desbloquea.

Los búferes constituyen una técnica muy utilizada; ésta también tiene una desventaja. Si los datos se colocan en búfer demasiadas veces, el rendimiento se reduce. Por ejemplo, considere la red de la figura que se muestra a continuación.

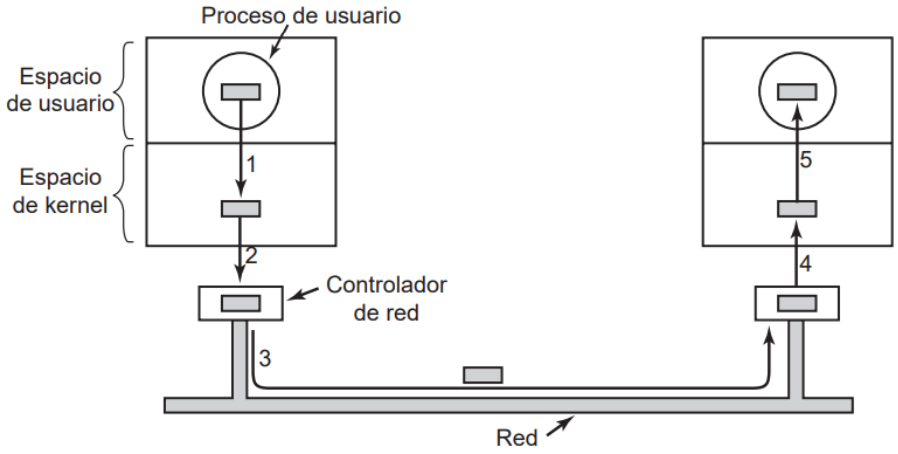


Ilustración 18 - El trabajo en red puede involucrar muchas copias de un paquete.

Aquí un usuario realiza una llamada al sistema para escribir en la red. El kernel copia el paquete a un búfer de kernel para permitir que el usuario proceda de inmediato (paso 1). En este punto, el programa de usuario puede reutilizar el búfer.

Cuando se hace la llamada al controlador, copia el paquete al controlador para enviarlo como salida (paso 2). La razón por la que no envía directamente al cable desde la memoria kernel es que, una vez iniciada la transmisión del paquete, debe continuar a una velocidad uniforme. El controlador no puede garantizar que pueda llegar a la memoria a una velocidad uniforme, debido a que los canales de DMA y otros dispositivos de E/S pueden estar robando muchos ciclos. Si no se obtiene una palabra a tiempo se podría arruinar el paquete. Al colocar en búfer el paquete dentro del controlador se evita este problema.

Una vez que se ha copiado el paquete en el búfer interno del controlador, se copia hacia la red (paso 3). Los bits llegan al receptor poco después de haber sido enviados, por lo que justo después de enviar el último bit, éste llega al receptor, donde el paquete se ha colocado en un búfer en el controlador.

A continuación, el paquete se copia al búfer del kernel del receptor (paso 4). Por último, se copia al búfer del proceso receptor (paso 5); generalmente, el receptor envía después un reconocimiento.

Cuando el emisor recibe el reconocimiento, puede enviar el segundo paquete. Sin embargo, debe quedar claro que todo este proceso de copia reduce la velocidad de transmisión de manera considerable, ya que todos los pasos se deben llevar a cabo en forma secuencial.

Reporte de errores.

Los errores son mucho más comunes en el contexto de la E/S que en otros. Cuando ocurren, el sistema operativo debe manejarlos de la mejor manera posible. Muchos errores son específicos de cada dispositivo y el driver apropiado debe manejarlos, pero el marco de trabajo para el manejo de errores es independiente del dispositivo.

Los errores de programación son una clase de errores de E/S. Éstos ocurren cuando un proceso pide algo imposible, como escribir en un dispositivo de entrada (teclado, escáner, ratón, etc.) o leer de un dispositivo de salida (una impresora o un plotter, por ejemplo). Otros errores son proporcionar una dirección de búfer inválida o algún otro parámetro, y especificar un dispositivo inválido (por ejemplo, el disco 3 cuando el sistema sólo tiene dos), entre otros. La acción a tomar en estos errores es simple: sólo se reporta un código de error al que hizo la llamada.

Otra clase de errores son los de E/S reales; por ejemplo, tratar de escribir un bloque de disco dañado o tratar de leer de una cámara de video que está apagada. En estas circunstancias depende del driver determinar qué hacer. Si el driver no sabe qué hacer, puede pasar el problema de vuelta al software independiente del dispositivo.

Lo que hace este software depende del entorno y la naturaleza del error. Si se trata de un error simple de lectura y hay un usuario interactivo disponible, puede mostrar un cuadro de diálogo pidiendo al usuario lo que debe hacer. Las opciones pueden incluir volver a intentar cierto número de veces, ignorar el error o eliminar el proceso que hizo la llamada. Si no hay usuario disponible, tal vez la única opción real sea hacer que la llamada al sistema falle con un código de error.

Sin embargo, algunos errores no se pueden manejar de esta forma. Por ejemplo, una estructura de datos crítica, como el directorio raíz o la lista de bloques libres, puede haberse destruido. En este caso, el sistema tal vez tenga que mostrar un mensaje de error y terminar.

Clase de Error	Origen	¿Qué ocurre?
De Programación	Software	El programa pide algo imposible (como escribir datos en un mouse).
De E/S Reales	Hardware	El dispositivo físico falla (como un disco dañado o una cámara apagada).

Asignación y liberación de dispositivos dedicados.

Algunos dispositivos, como los grabadores de CD-ROM, sólo pueden ser utilizados por un solo proceso en un momento dado. Es responsabilidad del sistema operativo examinar las peticiones de uso de los dispositivos y aceptarlas o rechazarlas, dependiendo de si el dispositivo solicitado está o no disponible. Una manera simple de manejar estas peticiones es requerir que los procesos realicen llamadas a open en los archivos especiales para los

dispositivos directamente. Si el dispositivo no está disponible, la llamada a open falla. Al cerrar un dispositivo dedicado de este tipo se libera.

Un método alternativo es tener mecanismos especiales para solicitar y liberar dispositivos dedicados.

Un intento por adquirir un dispositivo que no está disponible bloquea al proceso que hizo la llamada, en vez de fallar. Los procesos bloqueados se ponen en una cola. Tarde o temprano, el dispositivo solicitado estará disponible y el primer proceso en la cola podrá adquirirlo para continuar su ejecución.

Tamaño de bloque independiente del dispositivo.

Los distintos discos pueden tener diferentes tamaños de sectores. Es responsabilidad del software independiente del dispositivo ocultar este hecho y proporcionar un tamaño de bloque uniforme a los niveles superiores; por ejemplo, al tratar varios sectores como un solo bloque lógico. De esta forma, los niveles superiores lidian sólo con dispositivos abstractos que utilizan el mismo tamaño de bloque lógico, sin importar el tamaño del sector físico. De manera similar, algunos dispositivos de carácter envían sus datos un byte a la vez (como los módems), mientras que otros envían sus datos en unidades más grandes (como las interfaces de red). Estas diferencias también se pueden ocultar.

Software de E/S en espacio de usuario.

Aunque la mayor parte del software de E/S está dentro del sistema operativo, una pequeña porción de éste consiste en bibliotecas vinculadas entre sí con programas de usuario, e incluso programas enteros que se ejecutan desde el exterior del kernel. Las llamadas al sistema, incluyendo las llamadas al sistema de E/S, se realizan comúnmente mediante procedimientos de biblioteca.

Espacio	Componentes	¿Qué hace?
Espacio de Usuario *(User Space)*	Bibliotecas del sistema (como `libc`).	**Organiza:** Prepara los datos de los programas y les da un formato simple antes de pedirle ayuda al núcleo.
Espacio de Kernel *(Kernel Space)*	Drivers de dispositivos y manejadores de interrupciones.	**Controla:** Habla directamente con los circuitos del hardware y gestiona los búferes en la memoria principal.

Cuando un programa en C contiene la siguiente llamada:

```
cuenta = write(da, bufer, nbytes);
```

El procedimiento de biblioteca ***write*** se vinculará con el programa y se incluirá en el programa binario presente en memoria en tiempo de ejecución. La colección de todos estos procedimientos de biblioteca podría ser considerada parte del sistema de E/S.

Aunque estos procedimientos hacen algo más que colocar sus parámetros en el lugar apropiado para la llamada al sistema, hay otros procedimientos de E/S que en realidad realizan un trabajo real. En especial, el formato de la entrada y la salida se lleva a cabo mediante procedimientos de biblioteca.

Un ejemplo de C es ***printf***, que toma una cadena de formato y posiblemente unas variables como entrada, construye una cadena ASCII y después llama al sistema ***write*** para imprimir la cadena.

Como ejemplo de ***printf***, consideremos la siguiente instrucción:

```
printf(“El cuadrado de %3d es %6d\n”, i, i*i);
```

Esta instrucción da formato a una cadena que consiste en la cadena de 14 caracteres “El cuadrado de” seguida por el valor *i* como una cadena de 3 caracteres, después la cadena de 4 caracteres “es”, luego *i*2 como seis caracteres, y por último un salto de línea.

Un ejemplo de un procedimiento similar para la entrada es ***scanf***, que lee los datos de entrada y los almacena en variables descritas en una cadena de formato que utiliza la misma sintaxis que ***printf***.

La ***biblioteca de E/S estándar*** (stdio.h) contiene varios procedimientos que involucran operaciones de E/S y todos se ejecutan como parte los programas de usuario.

No todo el software de E/S en este nivel consiste en procedimientos de biblioteca. Otra categoría importante es el sistema de colas.

El ***uso de colas*** (*spooling*) es una manera de lidiar con los dispositivos de E/S dedicados en un sistema de multiprogramación.

Consideremos un dispositivo común que utiliza colas: una impresora. Aunque sería técnicamente sencillo dejar que cualquier proceso de usuario abriera el archivo de caracteres especial para la impresora, supongamos que un proceso lo abriera y no hiciera nada durante horas. Ningún otro proceso podría imprimir nada.

En vez de ello, lo que se hace es crear un proceso especial, conocido como ***demonio***, y un directorio especial llamado ***directorio de cola de impresión***. Para imprimir un archivo, un proceso genera primero todo el archivo que va a imprimir y lo coloca en el directorio de la cola de impresión.

Es responsabilidad del demonio, que es el único proceso que tiene permiso para usar el archivo especial de la impresora, imprimir los archivos en el directorio. Al proteger el archivo especial contra el uso directo por parte de los usuarios, se elimina el problema de que alguien lo mantenga abierto por un tiempo innecesariamente extenso.

Normalmente, este ***directorio de cola de impresión*** es una carpeta común y corriente dentro del sistema de archivos, administrada bajo reglas específicas por el sistema operativo.

Por ejemplo, en los sistemas basados en ***UNIX*** como ***Linux***, esa carpeta suele ser ***/var/spool/cups/*** o ***/var/spool/lpd/***.

El concepto se refleja de forma similar en Windows. Lo que cambia es el nombre de los componentes técnicos y la ruta de la carpeta en el almacenamiento.

En Windows, este sistema (dominio) se conoce como el ***Servicio de Cola de Impresión*** (Print Spooler). La Carpeta (directorio de cola) es una carpeta del sistema ubicada por defecto en la ruta ***C:\Windows\System32\spool\PRINTERS\.***

Sistema Operativo	¿Dónde está la carpeta? (Buzón)	¿Quién es el proceso de fondo?
Linux / UNIX	`/var/spool/cups/` o `/var/spool/lpd/`	Demonio (`cupsd` o `lpd`)
Windows	`C:\Windows\System32\spool\PRINTERS\`	Servicio (`spoolsv.exe`)

El uso de colas no es exclusivo de las impresoras. También se utiliza en otras situaciones de E/S. Por ejemplo, la transferencia de archivos a través de una red utiliza con frecuencia un demonio de red. Para enviar un archivo a cierta parte, un usuario lo coloca en un directorio de la cola de red. Más adelante, el demonio de red lo toma y lo transmite.

Directorio	Servicio / Demonio	¿Qué archivos guarda en espera?
`/var/spool/cups/`	`cupsd` (Impresión)	Documentos e imágenes listos para imprimir.
`/var/spool/postfix/`	`postfix` (Red / Email)	Correos salientes en cola o demorados.
`/var/spool/cron/crontabs/`	`crond` (Tareas)	Agendas de tareas automatizadas por cada usuario.

En la figura siguiente se resume el sistema de E/S, donde se muestran todos los niveles y las funciones principales de cada nivel.

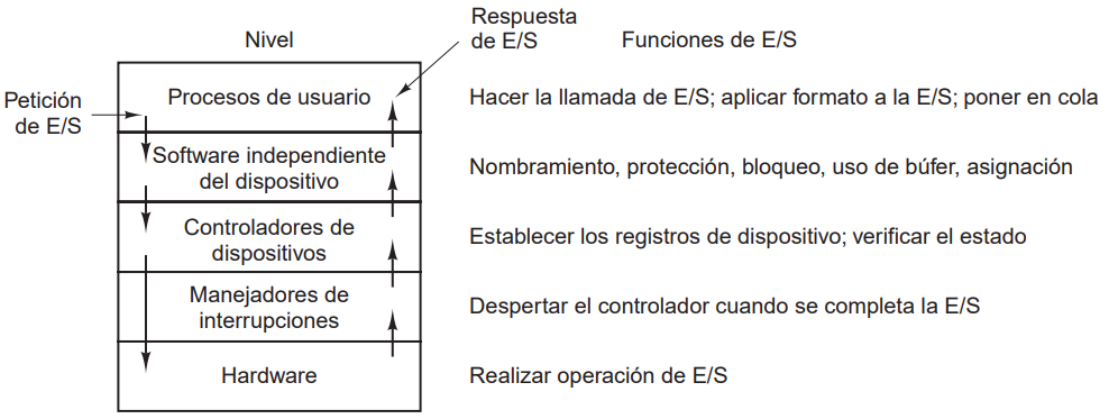


Ilustración 19 - Niveles del sistema de E/S y las funciones principales de cada nivel.

Empezando desde la parte inferior, los niveles son el hardware, los manejadores de interrupciones, los controladores de dispositivos, el software independiente del dispositivo y, por último, los procesos de usuario.

Las flechas en la figura muestran el flujo de control. Por ejemplo, cuando un programa de usuario trata de leer un bloque de un archivo, se invoca el sistema operativo para llevar a cabo la llamada. El software independiente del dispositivo busca el bloque en la caché del búfer, por ejemplo.

Si el bloque necesario no está ahí, llama al controlador del dispositivo para enviar la petición al hardware y obtenerlo del disco. Después, el proceso se bloquea hasta que se haya completado la operación de disco.

Cuando termina el disco, el hardware genera una interrupción. El manejador de interrupciones se ejecuta para descubrir qué ocurrió; es decir, qué dispositivo desea atención en ese momento. Después extrae el estado del dispositivo y despierta al proceso inactivo para que termine la petición de E/S y deje que el proceso de usuario continúe.

Referencias y bibliografía de la sección.

- Tanenbaum 5-3.

Control de energía.

La primera computadora electrónica de propósito general, ***ENIAC***, tenía 18,000 tubos al vacío y consumía 140,000 watts de energía.

Después de la invención del transistor, el uso de la energía disminuyó en forma considerable y la industria de las computadoras perdió el interés en los requerimientos de energía. Sin embargo, hoy en día la administración de la energía vuelve a estar en la mira por varias razones, y el sistema operativo desempeña un papel aquí.

Vamos a empezar con las PCs de escritorio. A menudo, una PC de escritorio tiene una fuente de energía de 200 watts (que por lo general tiene una eficiencia de 85%; es decir, pierde el 15% de la energía entrante debido al calor). Si se encienden 100 millones de estas máquinas al mismo tiempo en todo el mundo, en conjunto utilizan 20,000 megawatts de electricidad. Ésta es la producción total de 20 plantas de energía nuclear de un tamaño promedio.

Si se pudiera reducir a la mitad el requerimiento de energía, podríamos deshacernos de 10 plantas nucleares. Desde el punto de vista ambiental, deshacerse de 10 plantas de energía nuclear (o un número equivalente de plantas de combustible fósil) es un gran avance y bien vale la pena tratar de lograrlo.

El otro sitio donde la energía es una cuestión importante es en las computadoras operadas por batería, incluyendo las notebooks y las tablets. El núcleo del problema es que las baterías no pueden contener suficiente carga para durar mucho tiempo, cuando mucho alcanzan unas horas.

El sistema operativo desempeña un importante papel acá. En el nivel más bajo, los distribuidores de hardware están tratando de hacer que sus componentes electrónicos sean más eficientes en su uso de energía. Las técnicas utilizadas incluyen la reducción del tamaño de los transistores, el empleo de escalas de voltaje dinámicas, el uso de buses adiabáticos con poca desviación y técnicas similares.

Hay dos métodos generales para reducir el consumo de energía. El primero es que el sistema operativo apague partes de la computadora (en su mayoría, dispositivos de E/S) que no estén en uso, debido a que un dispositivo que está apagado utiliza menos energía (o nada).

El segundo método es que el programa de aplicación utilice menos energía, lo que posiblemente degradaría la calidad de la experiencia del usuario, para poder alargar el tiempo de la batería.

Cuestiones de hardware.

Las baterías son de dos tipos generales: desechables y recargables. Las baterías desechables (entre las más comunes se cuentan las AAA, AA y D) se pueden utilizar para operar dispositivos de bolsillo, pero no tienen suficiente energía como para operar computadoras notebook con grandes pantallas brillantes.

Por el contrario, una batería recargable puede almacenar suficiente energía como para operar una notebook durante unas cuantas horas. Las baterías de níquel-cadmio solían dominar esta área, pero cedieron el paso a las baterías híbridas de níquel-metal, que duran más tiempo y no contaminan tanto el ambiente cuando se desechan. Las baterías de ion-litio son aún mejores, y se pueden recargar sin tener que drenarse por completo primero, pero su capacidad también es muy limitada.

El método general que la mayoría de los distribuidores de computadoras utilizan para conservar las baterías es diseñar la CPU, la memoria y los dispositivos de E/S para que tenga múltiples estados: encendido, inactivo, hibernando y apagado.

Para utilizar el dispositivo, debe estar encendido. Cuando el dispositivo no se va a utilizar durante un tiempo corto se puede poner en inactividad, lo cual reduce el consumo de energía. Cuando no se va a utilizar durante un intervalo mayor, se puede poner en hibernación, lo cual reduce el consumo de energía aún más. La concesión aquí es que para sacar a un dispositivo de hibernación a menudo se requiere más tiempo y energía que para sacarlo del estado inactivo.

Por último, cuando un dispositivo se apaga, no hace nada y no consume energía. No todos los dispositivos tienen estos estados, pero cuando los tienen es responsabilidad del sistema operativo administrar las transiciones de estado en los momentos apropiados.

Algunas computadoras tienen dos o incluso tres opciones de estado/consumo de energía. Uno de estos puede poner a toda la computadora en estado inactivo, del que se puede despertar rápidamente al teclear un carácter o mover el mouse; otro botón puede poner a la computadora en estado de hibernación, del cual para despertarse requiere más tiempo. En ambos casos, estas opciones, no hacen nada más que enviar una señal al sistema operativo, que se encarga del resto en el software.

En algunos países los dispositivos eléctricos deben, por ley, tener un interruptor de energía mecánico que interrumpa un circuito y retire la energía del dispositivo, por razones de seguridad. Para cumplir con esta ley, tal vez sea necesario otro interruptor.

La administración de la energía hace que surjan varias preguntas con las que el sistema operativo debe lidiar. Muchas de ellas se relacionan con la hibernación de los recursos (apagar los dispositivos en forma selectiva y temporal, o al menos reducir su consumo de energía mientras están inactivos). Las preguntas que se deben responder incluyen las siguientes: ¿Qué dispositivos se pueden controlar? ¿Están encendidos/apagados, o tienen estados intermedios? ¿Cuánta energía se ahorra en los estados de bajo consumo de energía? ¿Se gasta energía al reiniciar el dispositivo? ¿Debe guardarse algún contexto cuando se pasa a un estado de bajo consumo de energía? ¿Cuánto se requiere para regresar al estado de energía máxima?

Desde luego que las respuestas a estas preguntas varían de dispositivo en dispositivo, por lo que el sistema operativo debe ser capaz de lidiar con un rango de posibilidades.

Dispositivo	Li y colaboradores (1994)	Lorch y Smith (1998)
Pantalla	68%	39%
CPU	12%	18%
Disco duro	20%	12%
Módem		6%
Sonido		2%
Memoria	0.5%	1%
Otros		22%

Ilustración 20 - Consumo de energía de varias partes de una computadora notebook.

En el caso de las notebooks, los principales tres consumidores de energía suelen ser la pantalla, el disco duro (si es que tienen) y la CPU, en ese orden.

Cuestiones del sistema operativo.

El sistema operativo desempeña un papel clave en la administración de la energía. Controla todos los dispositivos, por lo que debe decidir cuál apagar y cuándo hacerlo. Si apaga un dispositivo y éste se llega a necesitar rápidamente, puede haber un molesto retraso mientras se reinicia. Por otra parte, si espera demasiado para apagar un dispositivo, la energía se desperdicia.

El truco es buscar ***algoritmos*** y heurística que permitan al sistema operativo tomar buenas decisiones sobre lo que se va a apagar y cuándo se debe hacer. El problema es que "buenas" es muy subjetivo. Un usuario puede encontrar aceptable que después de 30 segundos de no utilizar la computadora, se requieran 2 segundos para que ésta responda a una pulsación de teclas. A otro usuario le puede parecer muy molesto bajo las mismas condiciones. En la ausencia de entrada de audio, la computadora no puede distinguir entre estos dos usuarios.

Ahora veamos los dispositivos que gastan la mayor parte de la energía, para ver qué se puede hacer al respecto.

La pantalla.

Uno de los principales elementos consumidores de energía es la pantalla. Para obtener una imagen nítida y brillante, la pantalla debe tener luz posterior, y eso requiere una energía considerable.

Muchos sistemas operativos tratan de ahorrar energía acá al apagar la pantalla cuando no hay actividad durante cierto número de minutos. A menudo, el usuario puede decidir cuál va a ser el intervalo de desconexión, con lo cual se deja al usuario la elección entre tener la pantalla en blanco frecuentemente y agotar la batería rápidamente (y tal vez el usuario no desee tener que hacer esta elección). Apagar la pantalla es un estado de inactividad, debido a que se puede regenerar (a partir de la RAM de video) casi en forma instantánea, cuando se oprime una tecla o se mueve el dispositivo señalador.

El disco duro.

El disco duro requiere una energía considerable para mantenerse girando a una alta velocidad, incluso aunque no haya accesos. Muchas computadoras, en especial las notebooks, hacen que el disco deje de girar después de cierto número de segundos o minutos de inactividad. Cuando se necesita otra vez, se vuelve a arrancar. Por desgracia, un disco detenido está en hibernación en vez de inactividad, ya que se requieren unos cuantos segundos para hacer que vuelva a girar, lo cual produce retrasos considerables para el usuario.

Además, al reiniciar el disco se consume una energía adicional considerable. Como consecuencia, cada disco tiene un tiempo característico (Td) que representa su punto muerto, a menudo en el rango de 5 a 15 segundos. Supongamos que el siguiente acceso al disco se espera durante cierto tiempo t en el futuro. Si t < Td, se requiere menos energía para

mantener el disco girando que para apagarlo y después volver a encenderlo rápidamente. Si t > Td, la energía que se ahorra hace que valga la pena apagar el disco y volver a encenderlo mucho después.

Si se pudiera hacer una buena predicción (por ejemplo, con base en los patrones de acceso anteriores), el sistema operativo podría hacer buenas predicciones para apagar dispositivos y ahorrar energía. En la práctica, la mayoría de los sistemas son conservadores y sólo detienen el disco hasta después de unos cuantos minutos de inactividad.

Otra forma de ahorrar energía del disco es tener una caché de disco de un tamaño considerable en la RAM. Si se necesita un bloque que se encuentra en la caché, no hay que reiniciar un disco inactivo para satisfacer la lectura. De manera similar, si se puede colocar en el búfer de la caché una escritura en el disco, no hay que reiniciar a un disco detenido sólo para realizar la escritura. El disco puede permanecer apagado hasta que se llene la caché u ocurra un fallo en la lectura.

Otra forma de evitar arranques innecesarios del disco es que el sistema operativo mantenga informados a los programas acerca del estado del disco, enviándoles mensajes o señales. Algunos programas tienen escrituras discrecionales que se pueden omitir o retrasar. Por ejemplo, un procesador de palabras se puede configurar para que escriba el archivo que se está editando al disco cada cierto número de minutos. Si el procesador de palabras sabe que el disco está apagado en el momento en que normalmente escribiría el archivo, puede retrasar esta escritura hasta que el disco se vuelva a encender, o hasta que haya transcurrido cierto tiempo adicional.

La CPU.

La CPU también se puede administrar para ahorrar energía. La CPU de una notebook se puede poner en estado inactivo mediante software, con lo cual se reduce el uso de la energía a casi cero. Lo único que puede hacer en este estado es despertarse cuando ocurra una interrupción. Por lo tanto, cada vez que la CPU está inactiva, ya sea en espera de E/S o debido a que no tiene nada que hacer, pasa al estado inactivo.

En muchas computadoras hay una relación entre el voltaje de la CPU, el ciclo de reloj y el uso de la energía. A menudo el voltaje de la CPU se puede reducir mediante software, lo cual ahorra energía, pero también reduce el ciclo de reloj (aproximadamente en forma lineal).

En la figura siguiente se muestra una comparación entre la operación a máxima velocidad y máxima energía durante cierto intervalo de tiempo, y la operación a la mitad de la velocidad y un cuarto de la energía por un intervalo del doble de tiempo.

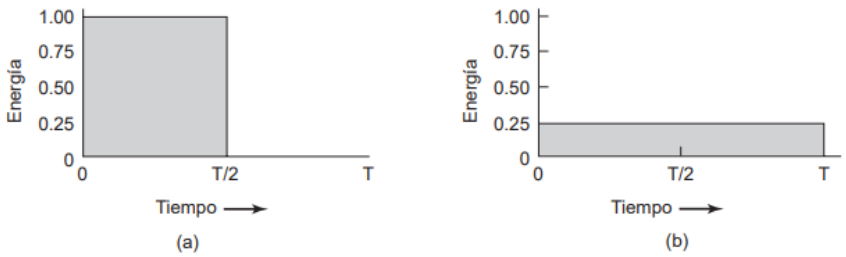


Ilustración 21 -. (a) Operación a máxima velocidad. (b) Recorte de voltaje por dos, de la velocidad por dos y del consumo de energía por cuatro.

En ambos casos se realiza el mismo trabajo, pero en la parte *b* de la figura sólo se consume la mitad de la energía al realizarlo.

La memoria.

Existen dos opciones posibles para ahorrar energía con la memoria. En primer lugar, la caché se puede vaciar y después apagarse. Siempre se puede volver a cargar de la memoria principal sin pérdida de información. La recarga se puede realizar en forma dinámica y rápida, por lo que apagar la caché es por completo un estado de inactividad.

Una opción más drástica es escribir el contenido de la memoria principal en el disco y después apagar la memoria principal en sí. Este método es la hibernación, ya que se puede cortar casi toda la energía a la memoria a expensas de un tiempo de recarga considerable, en especial si el disco también está apagado. Cuando se apaga la memoria, la CPU tiene que apagarse también, o tiene que ejecutarse de la ROM. Si la CPU está apagada, la interrupción que la despierta tiene que hacer que salte al código en la ROM, de manera que la memoria se pueda recargar antes de utilizarla.

A pesar de toda la sobrecarga, desconectar la memoria durante largos periodos (horas) puede valer la pena si es más conveniente reiniciar en unos cuantos segundos que tener que reiniciar el sistema operativo del disco, lo cual a menudo requiere de un minuto o más.

Administración térmica.

Una cuestión algo distinta, pero que también está relacionada con la energía, es la administración térmica. Las CPUs modernas se calientan en extremo debido a su alta velocidad. Los equipos de escritorio por lo general tienen un ventilador eléctrico interno para sacar el aire caliente del chasis.

Como la reducción del consumo de energía comúnmente no es una cuestión preponderante con los equipos de escritorio, normalmente el ventilador está encendido todo el tiempo.

Con las notebooks, la situación es distinta. El sistema operativo tiene que monitorear la temperatura en forma continua. Cuando se acerca a la temperatura máxima permisible, el sistema operativo tiene que tomar una decisión. Puede encender el ventilador, que hace ruido y consume energía.

Administración de baterías.

En los viejos tiempos, una batería sólo proporcionaba corriente hasta que se agotaba, momento en el cual se detenía. Esto ya no es así; actualmente, las laptops utilizan baterías inteligentes que se pueden comunicar con el sistema operativo. Si se les solicita, pueden informar su máximo voltaje, el voltaje actual, la máxima carga, la carga actual, la máxima proporción de agotamiento, la proporción de agotamiento actual y mucho más. La mayoría de las computadoras notebook tienen programas que se pueden ejecutar para consultar y mostrar todos estos parámetros.

A las baterías inteligentes también se les puede instruir para que cambien varios parámetros operacionales bajo el control del sistema operativo.

Interfaz de drivers.

El sistema Windows tiene un mecanismo elaborado para realizar la administración de energía, conocido como ***ACPI*** (Advanced Configuration and Power Interface, Interfaz avanzada de configuración y energía). El sistema operativo puede enviar a cualquier controlador que cumpla con este mecanismo por medio de comandos para pedir que reporte las capacidades de sus dispositivos y sus estados actuales. Esta característica es muy importante cuando se combina con los dispositivos “plug and play”, ya que justo después de iniciarse, el sistema operativo ni siquiera sabe qué dispositivos hay presentes, y mucho menos sus propiedades con respecto al consumo de energía o a su capacidad de administración de la misma.

También puede enviar comandos a los controladores para pedirles que reduzcan sus niveles de energía (con base en las capacidades que detectó antes, desde luego). También hay cierto tráfico en el sentido contrario. En especial cuando un dispositivo como un teclado o un mouse detecta actividad después de un periodo de inactividad, ésta es una señal para el sistema de que debe regresar a la operación (casi) normal.

Cuestiones de los programas de aplicaciones.

Hasta ahora hemos analizado las formas en que el sistema operativo puede reducir el uso de energía por parte de varios tipos de dispositivos. Pero también hay otro método: indicar a los programas que utilicen menos energía, aun si esto significa proporcionar una experiencia más pobre al usuario (es mejor una experiencia más pobre que ninguna experiencia, cuando la batería se agota y las luces se apagan). Por lo general, esta información se pasa cuando la carga de la batería está por debajo de cierto valor de umbral. Después es responsabilidad de los programas decidir entre degradar el rendimiento para extender la vida de la batería, o mantener el rendimiento y arriesgarse a quedarse sin energía.

Una de las preguntas que surge aquí es acerca de cómo puede un programa degradar su rendimiento para ahorrar energía. Cuando no hay degradación se presenta la mejor información posible. Cuando hay degradación, la fidelidad (precisión) de la información que se presenta al usuario es peor de lo que hubiera podido ser.

Un ejemplo de esto podría ser un programa que reproduce video. En modo no degradado, reproduce 30 cuadros/segundo en resolución completa y a colores. Una forma de degradación es abandonar la información de color y mostrar el video en blanco y negro. Otra forma de

degradación es reducir la velocidad de los cuadros, que produce parpadeos y proporciona a la película una calidad inferior. Otra forma más de degradación es reducir el número de pixeles en ambas direcciones, ya sea reduciendo la resolución espacial o la imagen a mostrar. De esta forma se podría ahorra una buena cantidad de energía.

Referencias y bibliografía de la sección.

- Tanenbaum 5-8.